

ACT WITH INTENT: DISTILLING BEHAVIOR INTENT FOR VISION-LANGUAGE-ACTION MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Vision-Language-Action (VLA) models can turn multimodal context into robot actions, but their action decoders are still trained largely by behavior cloning. This supervises which motor command was demonstrated while leaving implicit the local objective served by the behavior under the instruction. Future-based supervision enriches action learning with frames, latent observations, trajectories, or motion representations, but these signals capture particular realizations of what may happen rather than the shared semantic objective of the forthcoming behavior. We propose Intention Distillation (INDI), which distills behavior-level intent into the action decoder. During training, a frozen teacher VLM interprets a demonstrated segment from the current observation, instruction, coarse action summary, and corresponding execution video. From its standard inputs, the deployed VLA recovers the resulting multimodal intent representation at an intermediate decoder layer and uses it to organize action prediction together with representations of how the behavior unfolds and what it achieves. On SimplerEnv-Bridge, INDI improves GR00T-N1.7 from 64.3% to 84.7%, and on RoboCasa Kitchen it improves the controlled GR00T-N1.7 baseline from 64.1% to 70.3%, with consistent gains on $\pi_{0.5}$ across both benchmarks. In real-world tasks, INDI improves average success from 62.0% to 68.7%, with gains of up to 12.0 pp on longer-horizon tasks. Further analyses show that the recovered latent is used by the decoder, captures behavior objective and execution progress, and organizes downstream predictions in an objective-dependent manner. These results show that action decoders benefit from explicitly modeling the semantic objective of the behavior they generate.

1 INTRODUCTION

Recent Vision-Language-Action (VLA) models and robot foundation models have advanced rapidly by scaling vision-language backbones, action decoders, and robot demonstrations (Zitkovich et al., 2023; Kim et al., 2025; Black et al., 2025b;a; Physical Intelligence et al., 2026; NVIDIA et al., 2025; NVIDIA, 2026; Gemini Robotics Team et al., 2025; Li et al., 2024a). These models show that Internet-scale visual and linguistic knowledge can be transferred to robot control, and that specialized action decoders can turn multimodal context into continuous motor commands. Yet action-level training remains largely imitation-based. Behavior cloning tells the decoder which motor command to reproduce for a given observation and instruction, but not what that behavior is supposed to achieve under the instruction. As a result, the decoder can learn a strong context-to-action mapping while leaving the purpose of each behavior implicit.

To go beyond action-only supervision, a growing line of work augments VLA policies with future-based supervision. Some methods use future scene states, including generated frames, subgoal images, and latent observations (Wu et al., 2024; Black et al., 2024; Hu et al., 2025; Zheng et al., 2025b). Others represent how the forthcoming interaction unfolds through trajectories, motion fields, point tracks, or structured world dynamics (Zhao et al., 2025; Zhang et al., 2025; Xu et al., 2024; Bharadhwaj et al., 2024). These signals provide valuable information about what the future may look like or how motion may unfold, but they supervise particular realizations of behavior rather than the objective those realizations serve. This distinction matters because similar stages of manipulation can serve different objectives across tasks, while the same objective can be realized through diverse executions. To give the forthcoming action sequence its meaning under the instruction, the action decoder must represent the local objective it serves. This objective guides

the decoder in translating the scene and instruction context encoded by the VLA’s vision-language module into coherent, temporally extended behavior. We therefore argue that the decoder should recover a behavior-level intent that organizes action together with diverse representations of how the behavior unfolds and what it achieves.

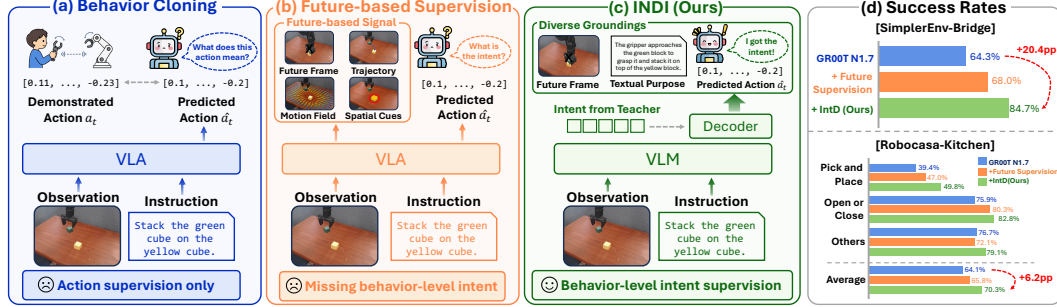


Figure 1: **From behavior cloning to intent distillation.** (a) Behavior cloning directly supervises actions. (b) Future-based supervision adds representations of future states or motion. (c) INDI distills behavior-level intent from executed behavior into the action decoder, where it organizes action and representations of how the behavior unfolds and what it achieves. Right: GR00T-N1.7 results on SimplerEnv-Bridge and RoboCasa.

To operationalize this principle, we propose Intention Distillation (INDI), which distills behavior-level intent into the action decoder. We define intent as the local objective that the forthcoming behavior should achieve under the instruction. During training, a frozen teacher VLM interprets an executed behavior segment from the current observation, instruction, a coarse action summary, and the execution video. The student decoder learns to recover the resulting multimodal intent representation from the current observation, instruction, and proprioceptive state, and uses it as an intermediate semantic state for action prediction.

Concretely, INDI realizes this behavior-level intent supervision through three components. **(1) Intent from executed behavior.** The teacher VLM identifies what object or relation changes, what local objective the segment serves, and how it advances the instruction. We use the teacher’s multimodal representation formed during this interpretation as the intent target, together with a generated purpose statement and an endpoint visual feature as complementary groundings. **(2) Intent recovery inside the action decoder.** Learnable intent queries recover the teacher target at an intermediate decoder layer, after which the remaining layers jointly complete action, latent visual and textual grounding predictions. **(3) Intent-aware decoding.** The recovered intent participates in action and grounding prediction, making it a functional intermediate representation rather than an auxiliary alignment target. At deployment, the teacher and all target-generation modules are removed.

We evaluate whether behavior-level intent improves action decoding and whether the recovered representation exhibits the expected structure of intent. On SimplerEnv-Bridge (Li et al., 2025b), INDI improves GR00T-N1.7 from 64.3% to 84.7%. On RoboCasa Kitchen (Nasiriany et al., 2024), it improves the controlled GR00T-N1.7 baseline from 64.1% to 70.3% across 24 tasks, with consistent gains on $\pi_{0.5}$ across both benchmarks. In real-world tasks, INDI improves success from 62.0% to 68.7% and generalizes to held-out objects and distractors. Representation analyses and closed-loop interventions further show that the recovered intent is used by the decoder, captures behavior objective and execution progress, and organizes downstream predictions in an objective-dependent manner. Together, these results show that behavior-level semantic supervision improves policy performance and generalization while inducing an internal representation that functions as intent.

Contributions. Our contributions are as follows:

- We identify behavior-level intent as a missing supervision target for VLA action decoders. Beyond imitating actions or predicting future states and motion, the decoder should model the local objective served by its action sequence.
- We propose INDI, which distills a teacher VLM’s multimodal understanding of executed behavior into intermediate action-decoder states. The recovered intent organizes action

prediction and representations of how the behavior unfolds and what it achieves, with no teacher-side modules at deployment.

- Across VLA backbones, SimplerEnv-Bridge (Li et al., 2025b), RoboCasa (Nasiriany et al., 2024), and real-world manipulation, INDI improves performance and OOD generalization. Analyses and interventions further show that the recovered latent functions as intent by encoding behavior objective and progress and organizing downstream predictions.

2 RELATED WORK

VLA models. VLA models combine large multimodal backbones with robot demonstrations to map observations and instructions to actions (Brohan et al., 2023; Zitkovich et al., 2023; Kim et al., 2025; Octo Model Team et al., 2024; Black et al., 2025b;a; Physical Intelligence et al., 2026; NVIDIA et al., 2025; NVIDIA, 2026; Gemini Robotics Team et al., 2025; Li et al., 2024a). Their progress has been supported by large robot datasets spanning diverse tasks, scenes, and embodiments (Open X-Embodiment Collaboration et al., 2024; Khazatsky et al., 2024; Walke et al., 2023; Fang et al., 2024). Recent systems increasingly pair pretrained vision-language modules with action decoders trained using flow-matching or diffusion objectives (Black et al., 2025b;a; NVIDIA et al., 2025; Li et al., 2024a; Liu et al., 2025; Wen et al., 2025; Kim et al., 2026a). While these advances improve action modeling, decoder supervision remains dominated by imitation of demonstrated motor commands. INDI instead supervises the local objective served by the action sequence.

Future-based and structured supervision. Prior work augments robot policies with generated future frames, subgoal images, and visual reasoning frames (Wu et al., 2024; Black et al., 2024; Zhao et al., 2025). Other methods learn predictive visual features, future latent states, world-action representations, or structured world knowledge as intermediate signals for action decoding (Hu et al., 2025; Tian et al., 2025; Zheng et al., 2025b; Zhang et al., 2025; Xu et al., 2026; Sun et al., 2026; Luo et al., 2026; Won et al., 2026). Related approaches represent spatial, motion, or semantic structure through optical flow, point tracks, visual traces, grounding masks, affordance chains, and embodied reasoning traces (Xu et al., 2024; Bharadhwaj et al., 2024; Zheng et al., 2025a; Huang et al., 2025b; Zawalski et al., 2025; Huang et al., 2025a; Li et al., 2025a). These methods demonstrate the value of intermediate supervision beyond motor commands, but define the decoder interface through a particular future state, motion structure, or reasoning representation. INDI instead supervises the behavior-level objective that organizes action and its diverse realizations.

Intent and latent plans in robot policies. Related methods derive deployment-time latent plans from trajectories, with Play-LMP learning continuous plans from play and LADS learning language-regularized discrete plans from trajectory segments (Lynch et al., 2020; Jiang et al., 2025). Their latents are reconstruction codes learned jointly with the policy, whereas INDI distills a fixed teacher’s semantic interpretation of the behavior. Prior work also operationalizes intent through trajectory abstractions, action priors, and virtual targets (Huang et al., 2026; Zhong et al., 2026; Pang et al., 2026), through temporal context or predicted scene structure (Lian et al., 2026; Chen et al., 2026; Fan et al., 2026; Xu et al., 2026), or through human cues such as demonstrations, gaze, and indirect instructions (Gupta et al., 2026; Xie et al., 2026; Tay et al., 2026; Li et al., 2026; Pani & Yang, 2026; Zuo et al., 2026; Chen et al., 2025). INDI instead defines intent as the local objective served by executed behavior under the instruction. A training-only teacher derives a multimodal intent target from the segment, distilled into intermediate decoder states and grounded in visual outcome and textual purpose. The deployed policy recovers it from standard VLA inputs.

3 METHODOLOGY

We propose INDI, which distills behavior-level intent into a pretrained VLA action decoder. During training, a frozen teacher VLM interprets multimodal evidence of the demonstrated behavior under the instruction, producing an intent target. The decoder recovers this target at an intermediate layer from its standard inputs and uses the recovered intent in subsequent action and visual and textual grounding prediction. The overall framework is illustrated in Figure 2.

3.1 PROBLEM FORMULATION

We consider a pretrained VLA policy with a frozen vision-language module $\mathcal{F}_{\text{vlm}}$ and a trainable flow-matching action decoder. At time t , the VLA receives an RGB observation o_t , a language instruction ℓ , and a proprioceptive state q_t . The VLM produces context tokens $B_t = \mathcal{F}_{\text{vlm}}(o_t, \ell)$.

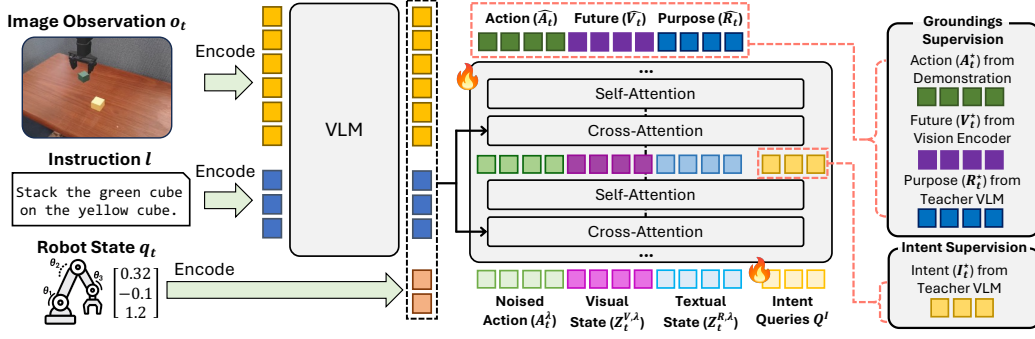


Figure 2: **Overview of INDI.** A training-only teacher VLM derives multimodal intent and textual-purpose targets from executed behavior, while a frozen visual encoder supplies the endpoint-visual target. The VLA decoder recovers intent at an intermediate layer and jointly predicts actions and latent visual and textual groundings, denoted by $\hat{V}_t$ and $\hat{R}_t$. Dashed lines indicate training-only supervision, and all target-generation modules are removed at deployment.

Our goal is to train the action decoder to predict A_t while forming an intermediate intent state I_t that represents the local objective served by the action under the instruction. Standard behavior cloning models $p_\theta(A_t | B_t, q_t)$, leaving this objective implicit. Let I_t denote the recovered intent state at the intermediate decoder layer. At each policy query, I_t is deterministically recovered from B_t and q_t , and the decoder models

$$(\hat{A}_t, \hat{V}_t, \hat{R}_t) \sim p_\theta(A_t, V_t, R_t | B_t, q_t, I_t). \quad (1)$$

Here, A_t , V_t , and R_t denote the action, visual-outcome, and textual-purpose supervision targets, while hats denote their decoder predictions.

3.2 INTENT SUPERVISION FROM EXECUTED BEHAVIOR

Since robot demonstrations do not explicitly label the local objective, we derive intent supervision using a frozen teacher VLM. For an H -step demonstrated behavior segment $\mathcal{W}_t$, let $c_A(\mathcal{W}_t)$ denote a coarse textual summary of its demonstrated actions. Under a functional-intent prompt, a teacher VLM processes $\mathcal{E}_t = (o_t, \ell, c_A(\mathcal{W}_t), \text{vid}_{t:t+H})$ and then generates a functional-purpose statement with token span $\mathcal{S}_t$. We cache the hidden states of both spans from the same autoregressive pass.

Target extraction. We extract the intent target from a middle teacher layer to retain both multimodal grounding and semantic abstraction before higher layers specialize toward language generation (Kim et al., 2026a). The textual-purpose target uses the final-layer representation of the generated statement, while the visual-outcome target uses a separate frozen encoding of the endpoint observation:

$$I_t^* = \text{pool}_{K_I}(T_{\mathcal{E}_t}^{(\ell_I)}), \quad R_t^* = \text{pool}_{K_R}(T_{\mathcal{S}_t}^{(\ell_R)}), \quad V_t^* = \text{pool}_{K_V}(\Phi_{\text{vis}}(o_{t+H})). \quad (2)$$

Here $T_{\mathcal{X}}^{(\ell)}$ denotes teacher hidden states over token span $\mathcal{X}$ at layer ℓ , and Φ_{vis} is the frozen visual encoder used for endpoint features. Thus, I_t^* retains the multimodal representation formed while the teacher interprets the executed behavior, whereas R_t^* provides its linguistic realization.

Visual and textual groundings. The endpoint visual target grounds intent in the resulting scene change, while the textual-purpose target grounds it in the functional meaning of the behavior. These choices build on prior work using predictive visual supervision (Wu et al., 2024; Black et al., 2024; Hu et al., 2025; Zhang et al., 2025) and language or reasoning traces for robot policies (Zawalski et al., 2025; Huang et al., 2025a; Li et al., 2025a; Sumers et al., 2023).

3.3 INTENT-AWARE ACTION DECODING

To make intent a functional intermediate state rather than an auxiliary readout, the decoder must recover it from the current VLA inputs before final action prediction and make it available to both action and grounding streams. We therefore augment the pretrained flow-matching action decoder with intent queries and visual and textual grounding rows. Let $Q_t = E_q(q_t)$ denote the proprioceptive-state representation, and let Q^I denote K_I clean learnable intent queries. The decoder processes

$$[Q_t, A_t^\lambda, Z_t^{V,\lambda}, Z_t^{R,\lambda}, Q^I],$$

where A_t^λ is the noised action input and $Z_t^{V,\lambda}$ and $Z_t^{R,\lambda}$ are self-conditioned visual and textual grounding inputs. The clean targets I_t^* , V_t^* , and R_t^* are used only for supervision.

Recovering intent. We supervise intent at an intermediate layer so that the recovered representation can shape the remaining decoder computation. Following intermediate representation alignment in VLA decoders (Zheng et al., 2025b), we read the contextualized intent-query states $H_{I,t}^{L_{\text{tap}}}$, project them into the teacher feature space, and optimize

$$\mathcal{L}_I = \frac{1}{K_I} \sum_{k=1}^{K_I} \left[1 - \cos \left(P_I(H_{I,t,k}^{L_{\text{tap}}}, \text{sg}(I_{t,k}^*)) \right) \right], \quad (3)$$

where P_I is a projection head and sg denotes stop-gradient. The decoder-space states $H_{I,t}^{L_{\text{tap}}}$ constitute the recovered intent used by the remaining layers.

Action and grounding prediction. The action stream retains the flow-matching objective of the pretrained VLA (Black et al., 2025a; NVIDIA et al., 2025). Given noise ϵ_A and flow time λ , let $A_t^\lambda = (1 - \lambda)\epsilon_A + \lambda A_t$ and $u_A = A_t - \epsilon_A$. The decoder predicts $\hat{u}_A$ using

$$\mathcal{L}_A = \mathbb{E}_{\lambda, \epsilon_A} \left[\|\hat{u}_A - u_A\|_2^2 \right]. \quad (4)$$

The grounding streams must remain available during decoding without receiving clean grounding targets that are unavailable at deployment. A no-gradient pass first predicts clean decoder-space grounding states $\bar{Z}_t^V$ and $\bar{Z}_t^R$. We detach and re-noise these predictions at the sampled flow time:

$$Z_t^{V,\lambda} = (1 - \lambda)\epsilon_V + \lambda \text{sg}(\bar{Z}_t^V), \quad Z_t^{R,\lambda} = (1 - \lambda)\epsilon_R + \lambda \text{sg}(\bar{Z}_t^R).$$

Rather than training separate flow generators in each target space, we align the final grounding states with their targets using representation alignment (Yu et al., 2025). For $G \in \{V, R\}$,

$$\mathcal{L}_G = \mathbb{E}_{\lambda, \epsilon_G} \left[\frac{1}{K_G} \sum_{k=1}^{K_G} \left(1 - \cos \left(P_G(H_{G,t,k}^{L_{\text{out}},\lambda}, \text{sg}(G_{t,k}^*)) \right) \right) \right]. \quad (5)$$

Action and grounding rows are jointly processed by the same decoder, allowing the recovered intent to organize their subsequent prediction.

Intent-dependent information flow. To make the recovered intent a functional intermediate state, we organize downstream decoding around it. All rows are processed jointly, but with asymmetric attention. Intent queries attend to Q_t , the VLA context B_t , and one another, while remaining isolated from the noised action and grounding rows. Grounding rows attend to the recovered intent and one another, whereas action rows attend to both intent and grounding rows. Furthermore, a binary gate α controls direct B_t access from non-intent rows. During training, we sample $\alpha \sim \text{Bernoulli}(0.5)$, and updates with $\alpha = 0$ require contextual information to pass through the intent queries.

For these intent-mediated updates, we further require downstream prediction to depend on the content of the recovered intent. We compare decoder continuations under the matched intent states $H_{I,t}^{L_{\text{tap}}}$ and batch-shifted states $\tilde{H}_{I,t}^{L_{\text{tap}}}$. Let $\mathcal{D}_{\text{right}}$ and $\mathcal{D}_{\text{swap}}$ denote the corresponding downstream action and grounding losses. We optimize

$$\mathcal{L}_{\text{mis}} = \max(0, m + \mathcal{D}_{\text{right}} - \mathcal{D}_{\text{swap}}), \quad (6)$$

which forces matched intent to yield lower downstream loss than mismatched intent by margin m .

Training objective and deployment. The full objective is

$$\mathcal{L} = \mathcal{L}_A + \lambda_I \mathcal{L}_I + \lambda_V \mathcal{L}_V + \lambda_R \mathcal{L}_R + \lambda_{\text{mis}} \mathcal{L}_{\text{mis}}. \quad (7)$$

At deployment, we set $\alpha = 1$ and remove the teacher, cached targets, alignment projections, and intent-mismatch branch. The policy receives the original VLA inputs and internally forms intent and grounding states together with the action stream.

Table 1: **Results on SimplerEnv-Bridge.** Success rates (%) on the four evaluated tasks. Controlled variants report mean $\pm$ sample standard deviation over three evaluation runs, while published baselines retain their originally reported values. Bold marks the best mean in each column, and underline marks the runner-up.

Method	Spoon	Carrot	Stack	EP-Basket	Avg.
<i>General-purpose VLA policies</i>					
Octo-base (Octo Model Team et al., 2024)	12.5	8.3	0.0	43.1	16.0
Octo-small (Octo Model Team et al., 2024)	47.2	9.7	4.2	56.9	29.5
OpenVLA (Kim et al., 2025)	0.0	0.0	0.0	4.1	1.0
RoboVLMs (Li et al., 2024b)	29.2	25.0	12.5	58.3	31.3
SpatialVLA (Qu et al., 2025)	16.7	25.0	29.2	100.0	42.7
π_0 (Black et al., 2025b)	46.7	38.7	42.7	39.3	41.8
π_0 -FAST (Pertsch et al., 2025)	59.0	79.0	<u>65.0</u>	33.0	59.0
GR00T-N1.5 (NVIDIA GEAR Team, 2025)	30.0	28.0	16.0	42.7	29.2
<i>Action abstraction and semantic reasoning</i>					
LAPA (Ye et al., 2025)	70.8	45.8	54.2	58.3	57.3
UniVLA (Bu et al., 2025)	52.8	55.6	2.8	80.6	47.9
ECOT (Zawalski et al., 2025; Zhang et al., 2026)	40.2	11.7	0.0	28.4	20.1
<i>Controlled backbone comparisons</i>					
GR00T-N1.7 (NVIDIA, 2026)	<u>84.7</u> ± 3.1	<u>79.3</u> ± 9.9	57.3 ± 8.1	36.0 ± 6.0	64.3 ± 3.3
GR00T-N1.7 + future supervision	81.3 ± 2.3	73.3 ± 4.2	56.7 ± 3.1	60.7 ± 4.2	<u>68.0</u> ± 1.0
GR00T-N1.7 + INDI (Ours)	88.7 ± 1.2	84.7 ± 6.1	69.3 ± 5.0	<u>96.0</u> ± 4.0	84.7 ± 0.8
Δ (Ours – baseline)	+4.0	+5.4	+12.0	+60.0	+20.4
$\pi_{0.5}$ (Black et al., 2025a)	78.0 ± 3.5	72.7 ± 7.6	32.0 ± 3.5	26.7 ± 4.2	52.3 ± 2.3
$\pi_{0.5}$ + INDI (Ours)	81.3 ± 4.2	76.0 ± 4.0	39.3 ± 5.0	38.7 ± 3.1	58.8 ± 3.3
Δ (Ours – baseline)	+3.3	+3.3	+7.3	+12.0	+6.5

4 EXPERIMENTS

We evaluate INDI across simulation and real-world manipulation settings to answer three questions: whether behavior-intent supervision improves policy performance across benchmarks and VLA backbones, whether its benefits extend to real-world and out-of-distribution conditions, and whether the recovered latent functions as intent rather than generic additional capacity. Our evaluation covers SimplerEnv-Bridge and RoboCasa Kitchen in simulation, together with real-world tabletop tasks involving held-out objects and distractors.

4.1 EXPERIMENTAL SETUP

Benchmarks. We evaluate INDI on two simulation benchmarks and a real-world manipulation setting. SimplerEnv-Bridge (Li et al., 2025b) covers four tabletop tasks, while RoboCasa Kitchen (Nasiriany et al., 2024) contains 24 household tasks spanning pick-and-place, articulated-object manipulation, and appliance interaction. Our real-world evaluation contains four tabletop tasks under standard scenes, held-out objects, and distractor conditions. Dataset, robot-platform, and evaluation-protocol details are provided in Section A.

Baselines. Our primary comparisons use GR00T-N1.7 (NVIDIA, 2026) and $\pi_{0.5}$ (Black et al., 2025a), with each baseline trained using the same demonstrations, optimization budget, and evaluation protocol as its corresponding INDI model. We additionally report published benchmark results for context. On RoboCasa Kitchen, GR00T checkpoints trained with 3,000 demonstrations per task are included only as data-scale references, rather than matched baselines. For real-world tasks, we compare against the GR00T-N1.7 under the same rollout protocol. Descriptions and comparison settings for all baselines are provided in Section A.3.

Implementation details. We retain each backbone’s optimizer, learning rate, and vision-language freezing configuration, and use the same INDI configuration across benchmarks unless stated otherwise. By default, we use $K_I = 8$, $K_R = 8$, $K_V = 16$ per camera view, an intent-alignment tap at 50% of the decoder depth, margin $m = 0.05$, context-dropout rate 0.5, and Cosmos-Reason2-8B (NVIDIA, 2025a) as the teacher VLM. Full hyperparameters, target-construction details, and implementation choices are provided in Section A.

4.2 RESULTS ON SIMULATION BENCHMARKS

We first test whether behavior-intent supervision improves pretrained VLA policies across distinct simulation benchmarks and backbone architectures.

Table 2: **Results on RoboCasa Kitchen.** Success rates (%) on the 24-task benchmark. G_n denotes n demonstrations per task. The G_{3000} checkpoints provide single-value data-scale references, while controlled G_{100} variants report mean $\pm$ sample standard deviation over three evaluation runs. Success rates are averaged within each task category, while Avg. is the macro-average across all 24 tasks. Bold marks the best mean in each column, and underline marks the runner-up.

Method	Pick-and Place	Open-or Close	Others	Avg.
GR00T-N1.6 (NVIDIA, 2025b) (G_{3000})	43.2	<u>81.0</u>	75.8	66.2
GR00T-N1.7 (NVIDIA, 2026) (G_{3000})	53.0	80.8	<u>79.0</u>	70.8
GR00T-N1.7 (G_{100})	39.4 $\pm$ 1.0	75.9 $\pm$ 3.4	76.7 $\pm$ 0.9	64.1 $\pm$ 1.6
GR00T-N1.7 + future supervision (G_{100})	47.0 $\pm$ 4.4	80.3 $\pm$ 2.3	72.1 $\pm$ 2.0	65.8 $\pm$ 0.8
GR00T-N1.7 + INDI (G_{100}, Ours)	49.8 $\pm$ 2.3	82.8 $\pm$ 1.3	79.1 $\pm$ 1.5	70.3 $\pm$ 1.7
Δ (Ours – baseline)	+10.4	+6.9	+2.4	+6.2
$\pi_{0.5}$ (Black et al., 2025a)	14.0 $\pm$ 1.7	55.1 $\pm$ 7.4	39.5 $\pm$ 0.9	34.9 $\pm$ 2.0
$\pi_{0.5}$ + INDI (Ours)	15.3 $\pm$ 1.4	56.1 $\pm$ 3.6	53.5 $\pm$ 2.4	41.4 $\pm$ 1.4
Δ (Ours – baseline)	+1.3	+1.0	+14.0	+6.5

SimplerEnv-Bridge. Table 1 reports success rates on four SimplerEnv-Bridge tasks. On GR00T-N1.7, INDI improves average success from 64.3% to 84.7%, a +20.4 pp gain. The improvement holds across all four tasks, including tasks where the baseline is already strong. The largest gain occurs on EP-Basket, where success increases from 36.0% to 96.0%, while Spoon, Carrot, and Stack improve by 4.0, 5.4, and 12.0 pp, respectively. Excluding EP-Basket, the remaining three tasks improve by 7.1 pp on average. The future-supervision variant reaches 68.0%, while INDI reaches 84.7%, a further +16.7 pp improvement. Among the reported results on this task suite, INDI achieves the highest average success. The improvement also transfers to $\pi_{0.5}$, increasing average success from 52.3% to 58.8%, a +6.5 pp gain, with improvements on every task.

RoboCasa Kitchen. Table 2 reports controlled comparisons on the 24-task RoboCasa Kitchen benchmark. With GR00T-N1.7 trained on 100 demonstrations per task, INDI improves average success from 64.1% to 70.3%, with gains across all three task categories. The future-supervision variant reaches 65.8%, whereas the full method reaches 70.3%, an additional +4.5 pp gain. The improvement again transfers to $\pi_{0.5}$, increasing average success from 34.9% to 41.4%, including a +14.0 pp gain on the Others.

Moreover, INDI trained with 100 demonstrations per task reaches 70.3% average success, within 0.5 pp of the reported GR00T-N1.7 checkpoint trained with 3,000 demonstrations per task. Table 3 further compares against reported RoboCasa Kitchen results from prior methods. Among these reported results, INDI achieves the highest average success rates. Together, the results show that INDI consistently improves two backbones across different simulation environments.

Table 3: **Reported RoboCasa Kitchen results.** Average success rates (%).

Method	Demos/task	Avg.
<i>Base VLA policies</i>		
GR00T-N1 (NVIDIA et al., 2025)	300	49.6
π_0 (Black et al., 2025b)	300	62.5
π_0 -FAST (Pertsch et al., 2025)	300	63.6
GR00T-N1.5 (NVIDIA GEAR Team, 2025)	300	65.7
<i>Future and video-based methods</i>		
DreamGen (Jang et al., 2025)	300	57.6
DUST (Won et al., 2026)	300	58.5
Video Policy (Liang et al., 2025)	300	66.0
FLARE (Zheng et al., 2025b)	300	66.4
<i>History and representation methods</i>		
HAMLET (Koo et al., 2026)	300	66.4
RS-CL (Kim et al., 2026b)	300	<u>69.7</u>
<i>Behavior-intent supervision</i>		
INDI (Ours)	100	70.3

4.3 RESULTS ON REAL-WORLD BENCHMARKS

Table 4 reports success rates on the four real-world tasks. INDI improves the baseline from 71.0% to 76.0% under ID clean, and the gain persists under held-out objects (62.0% to 68.0%) and distractors (53.0% to 62.0%). Relative retention is comparable under held-out objects (87.3% vs. 89.5%) and higher for INDI under distractors (74.6% vs. 81.6%), indicating that the improvement persists as the scene shifts from training conditions. The gains are concentrated in the longer tasks. Averaged across the three conditions, Cross-Bin Stacking and Drawer Storage improve by 7.3 and 10.7 pp, respectively, whereas Threading and Basket Nesting improve by 4.7 and 4.0 pp. Table 5 localizes these differences under ID clean. On Cross-Bin Stacking, both policies retrieve the base cube at the same rate (96.0%) and remain similar after centering (90.0% vs. 92.0%), but INDI retains higher completion when retrieving the top cube (82.0% to 86.0%) and completing the stack (74.0% to

Table 4: **Results on real-world tasks.** Success rates (%) over 50 trials. ID clean uses training objects without distractors, held-out substitutes unseen objects of the same functional role, and distractors add unrelated items to the scene. Bold marks the better result within each condition.

Task	ID clean		Held-out		Distractors	
	Base	+ INDI	Base	+ INDI	Base	+ INDI
Threading	92.0	96.0	86.0	84.0	70.0	82.0
Basket Nesting	94.0	92.0	84.0	90.0	76.0	84.0
Cross-Bin Stacking	74.0	80.0	62.0	72.0	58.0	64.0
Drawer Storage	24.0	36.0	16.0	26.0	8.0	18.0
Average	71.0	76.0	62.0	68.0	53.0	62.0

Table 6: **Controlled analyses on SimplerEnv-Bridge.** (a) compares supervision and capacity controls under the same backbone, demonstrations, budget, and evaluation run. Intent only removes the grounding streams, while future supervision replaces the teacher-intent target with an endpoint visual representation. (b) reports closed-loop success after replacing the task- or phase-discriminative coordinates of the intent and grounding representations. All values are success rates (%).

Method	Spoon	Carrot	Stack	EP-Basket	Avg.	Task	Injected objective				Forced phase		
							Spoon	Carrot	Stack	EP-Basket	Early	Late	Noise
GR00T-N1.7	82.0	68.0	66.0	30.0	61.5	Spoon	88.0	62.0	58.0	24.0	19.0	0.0	4.0
+ Groundings only	88.0	82.0	38.0	32.0	60.0	Carrot	50.0	84.0	56.0	32.0	6.0	18.0	0.0
+ Future supervision	84.0	70.0	56.0	62.0	68.0	Stack	48.0	42.0	72.0	36.0	4.0	4.0	0.0
+ Free latent	78.0	76.0	48.0	26.0	57.0	EP-Basket	42.0	48.0	44.0	94.0	0.0	0.0	0.0
+ Intent only	86.0	84.0	70.0	64.0	76.0	Avg. (diag/off-diag)	84.5 / 45.2				7.3	5.5	1.0
INDI (Ours)	90.0	78.0	74.0	100.0	85.5								

(a) Supervision controls.

(b) Objective and phase interventions.

80.0%). Drawer Storage exhibits a different pattern: the gap is already present at grasping (50.0% to 70.0%) and persists through the subsequent stages. The two shorter tasks remain near ceiling under ID clean and show only small, mixed differences.

The stage profiles reveal two different sources of improvement on the longer tasks. On Cross-Bin Stacking, the policies remain nearly indistinguishable through centering the base cube, and the main gap appears when execution must move from the completed first subgoal to retrieving and stacking the second cube. This provides the clearest real-world evidence that INDI improves behavior across a stage transition. Drawer Storage differs: INDI already improves grasping and transfer from 50.0% to 70.0%, and this advantage persists through opening (40.0% to 56.0%), placement (26.0% to 38.0%), and closure (24.0% to 36.0%). Thus, its gain cannot be attributed solely to a single transition, but instead reflects more reliable execution across the multi-stage sequence. By contrast, Threading and Basket Nesting have high early-stage completion and little separation under ID clean, leaving less room for improvement. Together, these results indicate that the benefit of intent supervision becomes more pronounced when successful execution must remain organized across multiple stages, while not requiring every gain to arise from the same failure mode.

Table 5: **Stage-level completion.** Fraction of trials (%) reaching each stage under ID clean (cumulative).

Task	Stage	Base	+ INDI
Threading	Pole centered	100.0	100.0
	Ring grasped	96.0	100.0
	Inserted	92.0	96.0
Basket Nest.	Lifted	100.0	96.0
	Nested	94.0	92.0
Cross-Bin	Base retrieved	96.0	96.0
	Base centered	90.0	92.0
	Top retrieved	82.0	86.0
	Stacked	74.0	80.0
Drawer	Grasped	50.0	70.0
	Transferred	50.0	70.0
	Opened	40.0	56.0
	Placed inside	26.0	38.0
	Closed	24.0	36.0

4.4 ANALYSIS AND CONTROLLED STUDIES

We isolate the source of the gain, test whether the recovered state functions as intent, and measure deployment cost. Unless stated otherwise, controlled analyses use GR00T-N1.7 on a fixed SimplerEnv-Bridge evaluation run shared across all variants. The main benchmark tables separately report means over three evaluation runs.

Is the gain specific to behavior-intent supervision? To distinguish teacher-derived intent from grounding supervision, future visual alignment, and additional latent capacity, we compare con-

trolled variants in Table 6a. Groundings only and free latent remain below the action-only baseline, while future supervision reaches 68.0%. Intent supervision alone reaches 76.0%, exceeding the baseline by 14.5 pp and future supervision by 8.0 pp. Adding visual and textual groundings further raises success to 85.5%, including a gain from 64.0% to 100.0% on EP-Basket. Thus, teacher-derived intent drives the main improvement, while groundings provide complementary gains on average. Further details are provided in Section B.1.

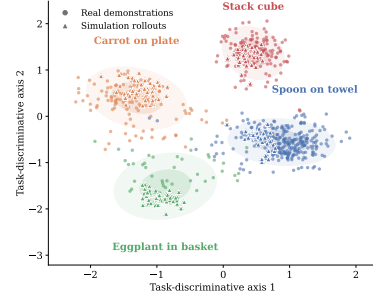
Does the recovered representation function as behavioral intent? The preceding controls identify intent supervision as the main source of the gain, so we next test whether the recovered state actually functions as intent by examining its objective and stage structure, its use by downstream decoding, and the behavioral effect of changing its content. As shown in Figure 3, recovered intents separate by behavior objective across real demonstrations and simulation rollouts and organize by early, middle, and late execution stages. Further analyses show that downstream decoding depends on the recovered state, quantify its objective and progress structure, and distinguish it from the particular action sequence used to realize the behavior (Section B.3, Table 12, and Fig. 6).

We next test whether this content causally affects execution by editing the task- or phase-discriminative components of the intent and grounding representations. Same-objective interventions retain 84.5% success, compared with 45.2% for cross-objective interventions and 1.0% under Gaussian corruption (Table 6b). The uneven cross-objective effects are consistent with recovered-intent task geometry, where Spoon and Carrot form the closest pair and EP-Basket is the most separated (Figure 7). Phase forcing reduces success to 7.3% for early content and 5.5% for late content, while producing stage-consistent behaviors such as approaching without grasping or attempting placement with an empty gripper (Figures 8 to 10). Together, these results show that recovered intent represents objective and progress beyond the particular action sequence and causally organizes downstream execution. Additional protocols and readouts are provided in Sections B.3 and B.4.

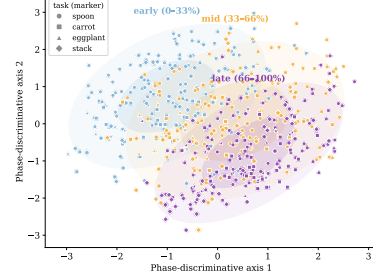
What is the additional computational cost? Teacher inference and target construction are performed once offline, and all teacher-side modules are removed at deployment, so runtime overhead comes only from the additional decoder representations. INDI increases GR00T-N1.7 from 3.46B to 3.50B parameters and inference time from 56.1 to 61.5 ms per policy query. For $\pi_{0.5}$, the corresponding changes are 3.62B to 3.64B parameters and 24.4 to 29.7 ms. Full training-time and resource measurements are reported in Section A.7 and Table 7.

5 CONCLUSION

We presented INDI, a framework for distilling behavior-level intent into pretrained VLA action decoders. While behavior cloning supervises executed actions and future-based objectives supervise particular realizations, neither directly identifies the local objective served by the behavior. During training, a frozen teacher VLM interprets demonstrated behavior, and the decoder learns to recover the resulting intent from standard VLA inputs and use it to organize action and grounding prediction. Across SimplerEnv-Bridge, RoboCasa Kitchen, and real-world manipulation tasks, INDI improves strong VLA backbones without requiring the teacher at deployment. Controlled studies further show that the gains arise specifically from intent supervision and that the recovered state represents objective and progress, is used by the policy, and influences closed-loop execution. These results position behavior intent as a compact intermediate representation that shifts VLA learning from reproducing executions toward understanding what each behavior should accomplish.



(a) **Objective structure.** A task-discriminative projection separates recovered intents by behavior objective across real and simulation rollouts.



(b) **Progress structure.** A phase-discriminative projection organizes intents from different tasks by execution stage.

Figure 3: **Semantic structure of recovered intent.** The representation retains both the local objective and progress through the manipulation sequence.

6 REPRODUCIBILITY STATEMENT

Appendix Section A documents the backbone integration, datasets, real-world platform, teacher prompt and target construction, implementation details, optimization settings, checkpoint selection, and computational resources. Appendix B provides the exact configurations and protocols for the controlled ablations, representation analyses, and closed-loop interventions, while Appendix C reports the full per-task results. Anonymous code, configuration files, target-construction scripts, and evaluation assets are provided in the supplementary material.

REFERENCES

- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2act: Predicting point tracks from internet videos enables diverse zero-shot robot manipulation. In *Euro-pean Conference on Computer Vision*, 2024.
- Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pre-trained image-editing diffusion models. In *International Conference on Learning Representations*, 2024.
- Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 17–40. PMLR, 2025a. URL <https://proceedings.mlr.press/v305/black25a.html>.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025b. doi: 10.15607/RSS.2025.XXI.010.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael S. Ryoo, Grecia Salazar, Pannag R. Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan H. Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. doi: 10.15607/RSS.2023.XIX.025.

- Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Learning to act anywhere with task-centric latent actions. In *Proceedings of Robotics: Science and Systems*, 2025.
- Yandu Chen, Kefan Gu, Yuqing Wen, Yucheng Zhao, Tiancai Wang, and Liqiang Nie. IntentionVLA: Generalizable and efficient embodied intention reasoning for human-robot interaction. *arXiv preprint arXiv:2510.07778*, 2025.
- Yi Chen, Yuying Ge, Hui Zhou, Mingyu Ding, Yixiao Ge, and Xihui Liu. Dial: Decoupling intent and action via latent world modeling for end-to-end vla. *arXiv preprint arXiv:2603.29844*, 2026.
- Liaoyuan Fan, Zetian Xu, Chen Cao, Wenyao Zhang, Mingqi Yuan, and Jiayu Chen. AIM: Intent-aware unified world action modeling with spatial value maps. *arXiv preprint arXiv:2604.11135*, 2026.
- Hao-Shu Fang, Hongjie Fang, Zhenyu Tang, Jirong Liu, Chenxi Wang, Junbo Wang, Haoyi Zhu, and Cewu Lu. RH20T: A comprehensive robotic dataset for learning diverse skills in one-shot. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 653–660. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10611615.
- Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, et al. Gemini robotics: Bringing ai into the physical world. *arXiv preprint arXiv:2503.20020*, 2025.
- Harsh Gupta, Guanya Shi, and Wenzhen Yuan. LUCID: Learning embodiment-agnostic intent models from unstructured human videos for scalable dexterous robot skill acquisition. *arXiv preprint arXiv:2606.11628*, 2026.
- Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 24328–24346. PMLR, 2025. URL <https://proceedings.mlr.press/v267/hu25g.html>.
- Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. ThinkAct: Vision-language-action reasoning via reinforced visual latent planning. In *Advances in Neural Information Processing Systems*, volume 38, 2025a.
- Haifeng Huang, Xinyi Chen, Yilun Chen, Hao Li, Xiaoshen Han, Zehan Wang, Tai Wang, Jiangmiao Pang, and Zhou Zhao. Roboground: Robotic manipulation with grounded vision-language priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 22540–22550, 2025b.
- Renming Huang, Chendong Zeng, Wenjing Tang, Jingtian Cai, Cewu Lu, and Panpan Cai. Mimic intent, not just trajectories. *arXiv preprint arXiv:2602.08602*, 2026.
- Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, Loïc Magne, Ajay Mandlekar, Avnish Narayan, You Liang Tan, Guanzhi Wang, Jing Wang, Qi Wang, Yinzhen Xu, Xiaohui Zeng, Kaiyuan Zheng, Ruijie Zheng, Ming-Yu Liu, Luke Zettlemoyer, Dieter Fox, Jan Kautz, Scott Reed, Yuke Zhu, and Linxi Fan. DreamGen: Unlocking generalization in robot learning through video world models. In Joseph Lim, Shuran Song, and Hae-Won Park (eds.), *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 5170–5194. PMLR, 2025. URL <https://proceedings.mlr.press/v305/jang25a.html>.
- Haobin Jiang, Jiangxing Wang, and Zongqing Lu. Discrete latent plans via semantic skill abstractions. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *Robotics: Science and Systems*, 2024.
- Dongyoung Kim, Huiwon Jang, Myungkyu Koo, Suhyeok Jang, Taeyoung Kim, Beomjun Kim, Byungjun Yoon, Changsung Jang, Daewon Choi, Dongsu Han, et al. RLDX-1 technical report. *arXiv preprint arXiv:2605.03269*, 2026a. URL <https://arxiv.org/abs/2605.03269>.

- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pp. 2679–2713. PMLR, 2025. URL <https://proceedings.mlr.press/v270/kim25c.html>.
- Taeyoung Kim, Jimin Lee, Myungkyu Koo, Dongyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. Contrastive representation regularization for vision-language-action models. In *International Conference on Machine Learning*, 2026b. URL <https://arxiv.org/abs/2510.01711>.
- Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. Hamlet: Switch your vision-language-action model into a history-aware policy. In *International Conference on Learning Representations*, 2026.
- Chengyang Li, Kaiyi Xiong, Yuan Xu, Lei Qian, Yizhou Wang, and Wentao Zhu. GazeVLA: Learning human intention for robotic manipulation. *arXiv preprint arXiv:2604.22615*, 2026.
- Jinming Li, Yichen Zhu, Zhibin Tang, Junjie Wen, Minjie Zhu, Xiaoyu Liu, Chengmeng Li, Ran Cheng, Yaxin Peng, and Feifei Feng. Coa-vla: Improving vision-language-action models via visual-text chain-of-affordance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025a.
- Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, Xiaofan Wang, Bei Liu, Jianlong Fu, Jianmin Bao, Dong Chen, Yuanchun Shi, Jiaolong Yang, and Baining Guo. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024a.
- Xinghang Li, Peiyan Li, Minghuan Liu, Dong Wang, Jirong Liu, Bingyi Kang, Xiao Ma, Tao Kong, Hanbo Zhang, and Huaping Liu. Towards generalist robot policies: What matters in building vision-language-action models. *arXiv preprint arXiv:2412.14058*, 2024b.
- Xuanlin Li, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl Pertsch, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pp. 3705–3728. PMLR, 2025b. URL <https://proceedings.mlr.press/v270/li25c.html>.
- Shijie Lian, Bin Yu, Xiaopeng Lin, Zhaolong Shen, Laurence Tianruo Yang, Yurun Jin, Haishan Liu, Changti Wu, Hang Yuan, Cong Huang, and Kai Chen. Intentvla: Short-horizon intent modeling for aliased robot manipulation. *arXiv preprint arXiv:2605.14712*, 2026.
- Junbang Liang, Pavel Tokmakov, Ruoshi Liu, Sruthi Sudhakar, Paarth Shah, Rares Ambrus, and Carl Vondrick. Video generators are robot policies. *arXiv preprint arXiv:2508.00795*, 2025.
- Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, 2025.
- Hao Luo, Wanpeng Zhang, Yicheng Feng, Sipeng Zheng, Haiweng Xu, Chaoyi Xu, Ziheng Xi, Yuhui Fu, and Zongqing Lu. Being-H0.7: A latent world-action model from egocentric videos. *arXiv preprint arXiv:2605.00078*, 2026.
- Corey Lynch, Mohi Khansari, Ted Xiao, Vikash Kumar, Jonathan Tompson, Sergey Levine, and Pierre Sermanet. Learning latent plans from play. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pp. 1113–1132. PMLR, 2020.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems*, 2024.

- NVIDIA. Cosmos-Reason2: Physical ai common sense and embodied reasoning models. GitHub repository, 2025a. URL <https://github.com/nvidia-cosmos/cosmos-reason2>. Released December 19, 2025.
- NVIDIA. Gr00t n1.6: An improved open foundation model for generalist humanoid robots. NVIDIA Research project page, December 2025b. URL https://research.nvidia.com/labs/gear/gr00t-n1_6/.
- NVIDIA. Nvidia isaac gr00t n1.7: A foundation model for generalist robots. GitHub repository, 2026. URL <https://github.com/NVIDIA/Isaac-GR00T>. Early access release.
- NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llonet, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- NVIDIA GEAR Team. GR00T N1.5: An improved open foundation model for generalist humanoid robots. NVIDIA Research project page, 2025. URL https://research.nvidia.com/labs/gear/gr00t-n1_5/.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Charles Xu, Jianlan Luo, Tobias Kreiman, and Sergey Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024. doi: 10.15607/RSS.2024.XX.090.
- Open X-Embodiment Collaboration, Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Alex Pooley, Aniruddha Gupta, Ajay Mandlekar, et al. Open x-embodiment: Robotic learning datasets and rt-x models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6892–6903. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10611477.
- Chuanke Pang, Junyi Huang, Zhijun Zhao, Yaobing Wang, Kun Xu, and Xilun Ding. Bridging the morphology gap: Adapting VLA models to dexterous manipulation via intent-conditioned fine-tuning. *arXiv preprint arXiv:2606.12109*, 2026.
- Anupam Pani and Yanchao Yang. Gaze-regularized vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2603.23202*, 2026.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Physical Intelligence. openpi: Open-source models and packages for robotics. <https://github.com/Physical-Intelligence/openpi>, 2025.
- Physical Intelligence, Bo Ai, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Danny Driess, Chelsea Finn, Karol Hausman, Brian Ichter, Sergey Levine, Karl Pertsch, Quan Vuong, et al. $\pi_{0.7}$: a Steerable Generalist Robotic Foundation Model with Emergent Capabilities. *arXiv preprint arXiv:2604.15483*, 2026.
- Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. Spatialvla: Exploring spatial representations for visual-language-action model. In *Robotics: Science and Systems*, 2025.
- Qwen Team. Qwen3.5-9B. Hugging Face model repository, 2026. URL <https://huggingface.co/Qwen/Qwen3.5-9B>.
- Theodore Sumers, Kenneth Marino, Arun Ahuja, Rob Fergus, and Ishita Dasgupta. Distilling internet-scale vision-language models into embodied agents. In *International Conference on Machine Learning*, 2023.

- Jingwen Sun, Wenyao Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. VLA-JEPA: Enhancing vision-language-action model with latent world model. *arXiv preprint arXiv:2602.10098*, 2026.
- Tracey Yee Hsin Tay, Xu Yan, Jonathan Ouyang, Daniel Wu, William Jiang, Jonathan Kao, and Yuchen Cui. Intent at a glance: Gaze-guided robotic manipulation via foundation models. *arXiv preprint arXiv:2601.05336*, 2026.
- Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, 2025.
- Homer Rich Walke, Kevin Black, Tony Z. Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, Abraham Lee, Kuan Fang, Chelsea Finn, and Sergey Levine. Bridgedata v2: A dataset for robot learning at scale. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pp. 1723–1736. PMLR, 2023.
- Junjie Wen, Minjie Zhu, Yichen Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Chengmeng Li, Xiaoyu Liu, Yaxin Peng, Chaomin Shen, and Feifei Feng. Diffusion-vla: Scaling robot foundation models via unified diffusion and autoregression. In *International Conference on Machine Learning*, 2025.
- John Won, Kyungmin Lee, Huiwon Jang, Dongyoung Kim, and Jinwoo Shin. Dual-stream diffusion for world-model augmented vision-language-action model. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2510.27607>.
- Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, 2024.
- Yifan Xie, YuAn Wang, Guangyu Chen, Jinkun Liu, Yu Sun, and Wenbo Ding. Learning human-intention priors from large-scale human demonstrations for robotic manipulation. *arXiv preprint arXiv:2604.24681*, 2026.
- Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Proceedings of the Conference on Robot Learning*, 2024.
- Xiaoxu Xu, Hao Li, Jinhui Ye, Yilun Chen, Jia Zeng, Xinyi Chen, Linning Xu, Dahua Lin, Weixin Li, and Jiangmiao Pang. FutureVLA: Joint visuomotor prediction for vision-language-action model. *arXiv preprint arXiv:2603.10712*, 2026.
- Seonghyeon Ye, Joel Jang, Byeongguk Jeon, Sejeun Joo, Jianwei Yang, Baolin Peng, Ajay Mandlekar, Reuben Tan, Yu-Wei Chao, Bill Yuchen Lin, Lars Liden, Kimin Lee, Jianfeng Gao, Luke Zettlemoyer, Dieter Fox, and Minjoon Seo. Latent action pretraining from videos. In *International Conference on Learning Representations*, 2025.
- Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion transformers is easier than you think. In *International Conference on Learning Representations*, 2025.
- Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pp. 3157–3181. PMLR, 2025.
- Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunan Wang, Xinqiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. Dreamvla: A vision-language-action model dreamed with comprehensive world knowledge. *arXiv preprint arXiv:2507.04447*, 2025.

- Yuchi Zhang, Churui Sun, Shiqi Liang, Diyuang Liu, Chao Ji, Wei-Nan Zhang, and Ting Liu. Bridging scale discrepancies in robotic control via language-based action representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 18809–18817, 2026.
- Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 1702–1713, June 2025.
- Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025a.
- Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, Fengyuan Hu, Joel Jang, Kaushil Kundalia, Zongyu Lin, Loïc Magne, Avnish Narayan, You Liang Tan, Guanzhi Wang, Qi Wang, Jiannan Xiang, Yinzen Xu, Seonghyeon Ye, Jan Kautz, Furong Huang, Yuke Zhu, and Linxi Fan. FLARE: Robot learning with implicit world modeling. In Joseph Lim, Shuran Song, and Hae-Won Park (eds.), *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 3952–3971. PMLR, 2025b. URL <https://proceedings.mlr.press/v305/zheng25a.html>.
- Linqing Zhong, Yi Liu, Yifei Wei, Ziyu Xiong, Maoqing Yao, Si Liu, and Guanghui Ren. ACoT-VLA: Action chain-of-thought for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. URL <https://arxiv.org/abs/2601.11404>.
- Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pp. 2165–2183. PMLR, 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.
- Kuangji Zuo, Gen Li, Bofan Lyu, Yanshuo Lu, Boyu Ma, Shijia Han, Xinyu Zhou, Xichen Yuan, Chuhao Zhou, Jiaqi Bai, Geng Li, and Jianfei Yang. Gaze2Act: Gaze-conditioned vision-language-action policies for interactive robot manipulation. *arXiv preprint arXiv:2605.30282*, 2026.

A EXPERIMENTAL DETAILS

A.1 MODEL DETAILS

Backbone VLAs. We instantiate INDI on two flow-matching VLA backbones, GR00T-N1.7-3B (NVIDIA, 2026) and $\pi_{0.5}$ (Black et al., 2025a; Physical Intelligence, 2025), using their official implementations. For both models, we freeze the pretrained vision-language module and fine-tune the action decoder together with the newly introduced INDI parameters.

INDI integration. For GR00T-N1.7, we append self-conditioned visual and textual grounding rows and clean intent queries Q^I to the pretrained DiT token stream. The grounding rows follow the backbone’s time-conditioned normalization, while the intent queries use standard layer normalization and are never noised. The recovered intent is supervised at block 16 of 32. For $\pi_{0.5}$, the

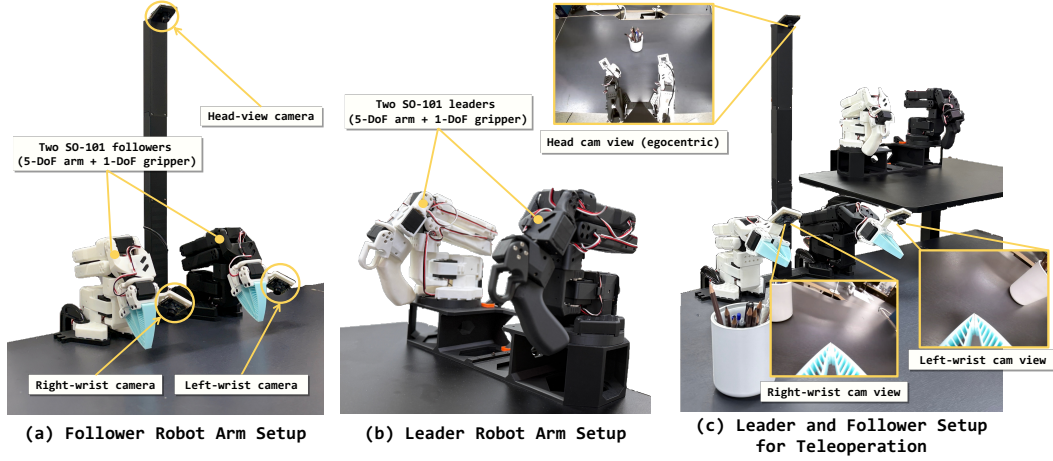


Figure 4: **Real-robot teleoperation and evaluation platform.** We specify the leader and follower robot arm setups, consisting of 5-DoF arms with 1-DoF grippers, along with the multi-camera views.

action-expert suffix is organized as

$$[Q_t, A_t^\lambda, Z_t^{V,\lambda}, Z_t^{R,\lambda}, Q^I],$$

where Q_t is the proprioceptive-state representation, A_t^λ is the noised action sequence, $Z_t^{V,\lambda}$ and $Z_t^{R,\lambda}$ are self-conditioned visual and textual grounding inputs, and Q^I contains the clean intent queries. The recovered intent is supervised at layer 9 of 18. We apply the same asymmetric information-flow pattern during training and sampling. Intent queries attend to the frozen VLA context, proprioceptive state, and one another while remaining isolated from the noised action and grounding rows. Visual and textual grounding rows attend to the recovered intent and one another, whereas action rows attend to the intent together with both grounding streams. For $\pi_{0.5}$, we cache the frozen prefix keys and values during training.

Trainable and frozen components. The vision-language modules, including their visual encoders, remain frozen for both backbones. We optimize the complete action decoder together with the added intent and grounding parameters. These parameters include intent-query embeddings, positional and camera-view embeddings, and three-layer SiLU MLPs for encoding and projecting the target representations. The visual target has width $d_V = 2048$, the teacher targets have width $d_T = 4096$, and the added representations are mapped to the backbone-specific decoder width d_D . Checkpoint-tensor counting gives 3.455B parameters for the GR00T-N1.7 baseline and 3.502B for GR00T-N1.7+INDI, corresponding to 46.4M additional parameters or a 1.3% increase. For $\pi_{0.5}$, the corresponding counts are 3.617B and 3.640B, giving 23.1M additional parameters or a 0.64% increase.

Model dimensions. Unless stated otherwise, we use $K_I = 8$ intent queries, $K_R = 8$ textual grounding rows, and $K_V = 16$ visual grounding rows per camera. Bridge uses one camera and therefore 16 visual rows, while RoboCasa Kitchen uses three cameras and therefore 48 visual rows. We use an intent-mismatch margin of $m = 0.05$ and a context-dropout probability of 0.5.

A.2 DATASETS

SimplerEnv-Bridge. We evaluate on SimplerEnv-Bridge (Li et al., 2025b), a simulation benchmark for evaluating real-world manipulation policies trained on BridgeData V2 (Walke et al., 2023). We use the LeRobot conversion of BridgeData V2, which contains 53,192 episodes and 1,893,026 frames recorded at 5 Hz. Each sample contains a primary RGB observation at 256×256 resolution, an 8-dimensional robot state, and a 7-dimensional action. We evaluate four WidowX manipulation tasks: spoon on towel, carrot on plate, stack cube, and put eggplant in basket. Our controlled GR00T-N1.7 and $\pi_{0.5}$ comparisons use three independent evaluation runs with 50 episodes per task in each run. We report the mean and sample standard deviation of the three run-level success rates. Published baseline results follow their original evaluation protocols.

RoboCasa Kitchen. RoboCasa Kitchen (Nasiriany et al., 2024) is a simulation benchmark for household manipulation in diverse kitchen environments. We use its machine-generated dataset with 100 demonstrations for each of 24 tasks, giving 2,400 episodes and 689,595 frames recorded at 20 Hz. Each sample contains left, right, and wrist RGB observations at 256×256 resolution, a 53-dimensional robot state, and a 12-dimensional action. The benchmark contains 8 pick-and-place tasks, 6 open-or-close tasks, and 10 additional appliance and interaction tasks. We use three independent evaluation runs with 50 episodes per task and run for both GR00T-N1.7 and $\pi_{0.5}$. Category and overall averages are computed within each run and then summarized by their mean and sample standard deviation across the three runs.

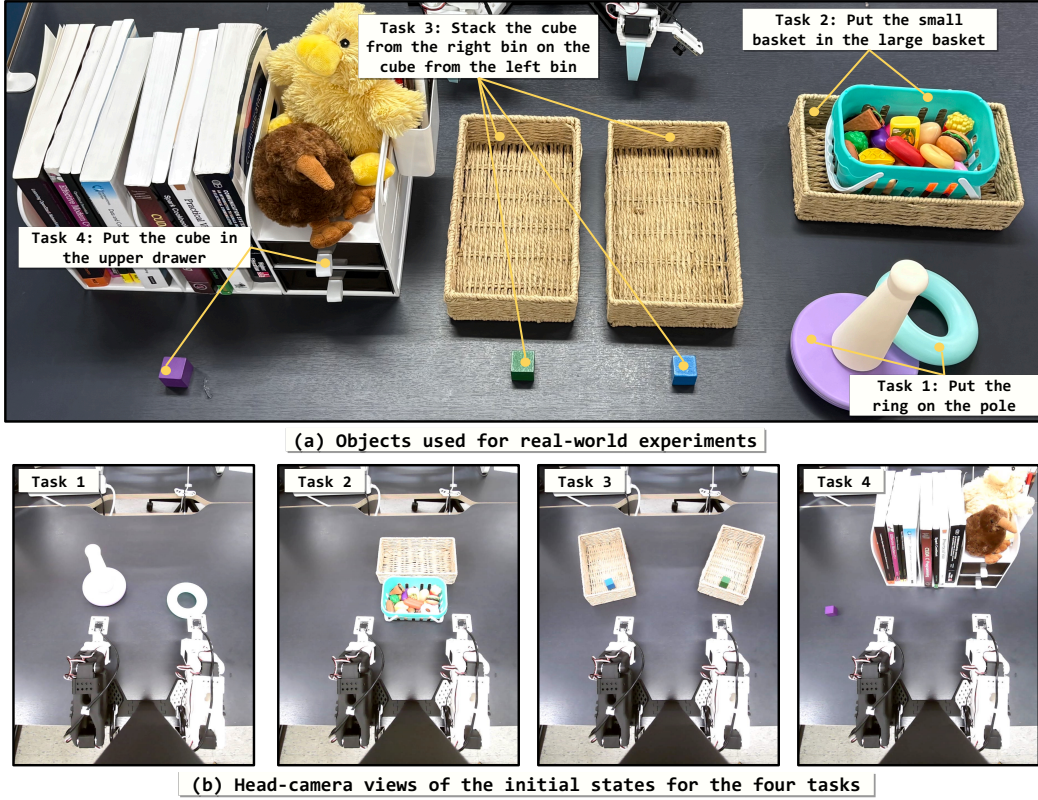


Figure 5: **Real-world task suite.** Four tabletop tasks used for real-world experiments.

Real-world platform. As shown in Figure 4, we use a teleoperation setup consisting of SO-101 robot arms for both the leader and follower, equipped with 1-DoF grippers. Three camera views are provided: an egocentric view mounted on the head structure and two local views on the wrists.

On this platform, we design four tabletop tasks as illustrated in Figure 5, each targeting a distinct capability and ordered by horizon length. For each task, we collect 100 demonstrations and train a single multitask policy per method on identical data.

- *Threading* (bimanual) has the left arm bring a pole to the center, after which the right arm inserts a specified ring onto it, testing precise alignment.
- *Basket Nesting* (bimanual) lifts a small basket with both arms into a larger one, testing simultaneous cooperation on a shared object that a single arm cannot lift.
- *Cross-Bin Stacking* (bimanual) has the left arm place a cube from the left bin at the center, after which the right arm stacks a cube from the right bin on top.
- *Drawer Storage* (bimanual) has the left arm pass an object to the right arm, which then opens the specified drawer of a two-tier unit, places the object inside, and closes it.

A.3 BASELINES

We briefly describe the baseline methods included in our evaluations.

- **Octo** (Octo Model Team et al., 2024) is a transformer-based generalist policy trained on cross-embodiment robot data with a diffusion-based action head.
- **OpenVLA** (Kim et al., 2025) fine-tunes a pretrained vision-language model to autoregressively predict discretized action tokens.
- **RoboVLMs** (Li et al., 2024b) studies the integration of pretrained vision-language models with continuous and discrete action-prediction architectures.
- **SpatialVLA** (Qu et al., 2025) incorporates explicit 3D spatial information through Ego3D positional encoding and adaptive action grids.
- π_0 (Black et al., 2025b) combines a pretrained vision-language prefix with a flow-matching action expert for continuous action generation.
- π_0 -**FAST** (Pertsch et al., 2025) predicts actions autoregressively using frequency-space action tokenization.
- $\pi_{0.5}$ (Black et al., 2025a) extends the π_0 family toward open-world generalization and serves as our second backbone.
- **GR00T-N1** (NVIDIA et al., 2025) combines a vision-language module with a flow-matching diffusion-transformer action decoder.
- **GR00T-N1.5** (NVIDIA GEAR Team, 2025) improves the architecture, training data, and post-training procedure of GR00T-N1.
- **GR00T-N1.6** (NVIDIA, 2025b) further updates the GR00T pretraining mixture and post-training recipe.
- **GR00T-N1.7** (NVIDIA, 2026) is our primary backbone.
- **LAPA** (Ye et al., 2025) learns discrete latent actions between video frames and pretrains a vision-language model to predict these representations before robot-action fine-tuning.
- **UniVLA** (Bu et al., 2025) learns task-centric latent actions from cross-embodiment videos and decodes the predicted latent actions into embodiment-specific robot trajectories.
- **ECot** (Zawalski et al., 2025) generates embodied reasoning traces before predicting robot actions.
- **DreamGen** (Jang et al., 2025) generates synthetic robot trajectories with a video world model and recovers corresponding pseudo-actions.
- **DUST** (Won et al., 2026) jointly predicts future observations and actions using interacting diffusion streams with decoupled flow-matching objectives.
- **Video Policy** (Liang et al., 2025) combines behavior-video generation and action prediction within an end-to-end robot policy.
- **FLARE** (Zheng et al., 2025b) aligns intermediate action-decoder representations with latent features of future observations.
- **HAMLET** (Koo et al., 2026) incorporates observation history through moment tokens and a temporal memory module.
- **RS-CL** (Kim et al., 2026b) regularizes vision-language representations with a contrastive objective supervised by distances between robot proprioceptive states.

Our controlled comparisons use GR00T-N1.7 and $\pi_{0.5}$ baselines trained with the same demonstrations, optimization budgets, and evaluation protocols as their corresponding INDI models. Moreover, future supervision replaces the teacher-intent target with FLARE-style alignment of mid-depth decoder representations to latent features of the future observation, under the same backbone and training budget. The exact configurations of the supervision and capacity controls are provided in [Section B.1](#).

A.4 TARGET CONSTRUCTION

Teacher inputs and prompt. We use the frozen Cosmos-Reason2-8B model (NVIDIA, 2025a) as the teacher. For each H -step demonstrated behavior segment $\mathcal{W}_t$, the teacher receives the current observation, language instruction, a coarse textual action summary $c_A(\mathcal{W}_t)$, and the corresponding execution video. In a single autoregressive generation pass, the teacher first processes this multimodal evidence and then generates a functional-purpose statement describing what the executed behavior accomplishes under the instruction. The prompt asks the teacher to reason in at most four sentences about the state change occurring during the segment and how it advances the instruction. It then requests exactly one sentence of approximately 30 to 50 tokens that describes the object-level state change or immediate local subgoal. Low-level control values, camera motion, references to the video, and additional labels or explanations are explicitly excluded. We use the same prompt template for Bridge and RoboCasa Kitchen.

Prompt for Teacher Target Construction

System Prompt

You are a robot manipulation reasoning model. You will see a short video of an approximately 1.6-second segment from a longer robot task, a short multi-step segment that may include contact, object motion, or clear setup for the next contact, plus a coarse summary of the commanded robot motion.

First write a brief reasoning block inside `<think>...</think>`, using at most four sentences. In the reasoning, infer what changes over this segment: which object or relation is acted on, what state transition occurs, and why this segment advances the episode instruction.

Immediately after `</think>`, write exactly one sentence of roughly 30–50 tokens. This sentence is the segment’s functional intent: the object-level outcome that the behavior accomplishes or clearly progresses toward. Describe the intended object-level state change, not low-level motion, camera appearance, or the video itself. State only changes that are visible or strongly implied by the segment. If contact or object motion has not happened yet, describe the immediate setup or alignment achieved by the segment.

Do not mention “the video,” “the clip,” “frames,” “I,” or “the robot intends.” Do not add a label, bullet, prefix, or explanation after `</think>`. Output only the one sentence after `</think>`.

User Prompt

Episode instruction: {instruction}

Coarse commanded motion over this segment: {action_summary}

The video frames show an approximately 1.6-second multi-step segment with {n} frames.

Identify the functional intent of this behavior segment by stating the object-level outcome it accomplishes or clearly progresses toward under the episode instruction.

Action and video preprocessing. We convert each continuous action segment into a compact textual representation before providing it to the teacher. For each action dimension, we compute dataset-specific percentile boundaries from the training set and map the continuous values to coarse discrete motion bins. The resulting sequence of bucketed values is serialized as a short description of the translational, rotational, and gripper behavior over the segment. The gripper dimension is interpreted using the open and close convention of each dataset rather than assuming a shared sign convention across embodiments. This representation provides the teacher with the overall motion pattern while avoiding long sequences of raw floating-point values. The execution video covers the same temporal interval as the summarized action segment. On Bridge, we use the 8 frames corresponding to an 8-step segment recorded at 5 Hz. On RoboCasa Kitchen, each target covers a 32-step segment recorded at 20 Hz, from which we uniformly sample 16 frames. Both therefore represent an approximately 1.6-second behavior window. The current observation, action segment, and video clip are matched using their episode identifier and segment start index, ensuring that all teacher inputs describe the same executed behavior.

Intent and textual-purpose targets. Let $\mathcal{E}_t$ denote the teacher-input span containing the current observation, instruction, coarse action summary, and execution video. Let $\mathcal{S}_t$ denote only the final functional-purpose statement generated after the reasoning span. During the same autoregressive pass, we cache the hidden states produced while processing $\mathcal{E}_t$ and while generating $\mathcal{S}_t$. We construct the intent target I_t^* from the layer-18 hidden states over $\mathcal{E}_t$, corresponding to the midpoint of the teacher. Given the input-span hidden states $H_{\mathcal{E}_t}^{(18)}$, we partition the token sequence into $K_I = 8$

approximately equal contiguous regions and average the hidden states within each region:

$$I_{t,k}^* = \frac{1}{|\mathcal{G}_{I,k}|} \sum_{i \in \mathcal{G}_{I,k}} H_{\mathcal{E}_t,i}^{(18)}, \quad k = 1, \dots, K_I.$$

This preserves the coarse ordering of the multimodal teacher-input representation while producing a fixed number of target slots. We construct the textual-purpose target R_t^* from the final-layer hidden states over $\mathcal{S}_t$. The statement tokens are likewise partitioned into $K_R = 8$ contiguous regions and mean-pooled:

$$R_{t,k}^* = \frac{1}{|\mathcal{G}_{R,k}|} \sum_{i \in \mathcal{G}_{R,k}} H_{\mathcal{S}_t,i}^{(\text{final})}, \quad k = 1, \dots, K_R.$$

Both targets have hidden width $d_T = 4096$. Thus, I_t^* is the evidence-side multimodal representation formed while the teacher interprets the demonstrated behavior, whereas R_t^* captures its final linguistic realization.

Generated-response intent target for the target-source ablation. We additionally construct an alternative intent target from the teacher-generated response. Let $\mathcal{Y}_t$ denote the complete response span generated after processing $\mathcal{E}_t$, including both the reasoning block and the final functional-purpose statement. Using the same teacher layer as the default intent target, we partition the hidden states over $\mathcal{Y}_t$ into $K_I = 8$ approximately equal contiguous regions and mean-pool each region:

$$I_{t,k}^{*,\text{resp}} = \frac{1}{|\mathcal{G}_{\text{resp},k}|} \sum_{i \in \mathcal{G}_{\text{resp},k}} H_{\mathcal{Y}_t,i}^{(18)}, \quad k = 1, \dots, K_I.$$

This alternative preserves the same target width and number of slots as the default evidence-side target, while replacing the representation formed over the multimodal evidence with one formed over the teacher’s complete generated interpretation.

Visual-outcome target. We construct the visual-outcome target V_t^* from the observation at the endpoint of the behavior segment. Specifically, the endpoint is o_{t+8} for Bridge and o_{t+32} for Robo-Casa Kitchen, matching the temporal window used for the corresponding teacher target. We encode this observation using the frozen visual encoder of Cosmos-Reason2-2B. The encoder produces 64 regional features with hidden width $d_V = 2048$, which we reduce to $K_V = 16$ ordered target slots using fixed 64-to-16 spatial pooling. For multi-camera observations, each view is encoded and pooled independently. Bridge uses one camera and therefore produces 16 visual target slots. Robo-Casa Kitchen uses left, right, and wrist cameras and therefore produces 48 slots in total. Samples that do not contain a valid endpoint observation near an episode boundary are excluded from the visual-alignment loss. All teacher and visual targets are generated once offline and reused across policy-training runs.

A.5 IMPLEMENTATION

Decoder augmentation. We preserve the native state and action representations of each backbone and append self-conditioned visual grounding rows $Z_t^{V,\lambda}$, self-conditioned textual grounding rows $Z_t^{R,\lambda}$, and clean intent queries Q^I to the action-decoder sequence. For GR00T-N1.7, the added rows are incorporated into the diffusion-transformer stream. For $\pi_{0.5}$, they are appended to the action-expert suffix following the frozen vision-language prefix. All added rows use learned positional embeddings, and visual rows additionally use camera-view embeddings. For $\pi_{0.5}$, we cache the frozen prefix keys and values during training.

Asymmetric information flow. Intent queries attend to the robot state, frozen VLA context, and one another, but not to the noised action or grounding rows. Visual and textual grounding rows attend to the recovered intent and one another, while action rows attend to the recovered intent and both grounding streams. Direct context access from action and grounding rows is controlled by a binary gate α , whereas intent queries always retain access to the VLA context. During training, we sample $\alpha \sim \text{Bernoulli}(0.5)$, so updates with $\alpha = 0$ require the downstream rows to obtain contextual information through the intent queries. For $\pi_{0.5}$, this gate is applied to the image and language prefix tokens, while access to the state representation remains available. We use the same information-flow structure during training and sampling and set $\alpha = 1$ at deployment.

Intent alignment. We extract the contextualized intent-query states $H_{I,t}^{L_{\text{tap}}}$ at the midpoint of the decoder, using block 16 for GR00T-N1.7 and layer 9 for $\pi_{0.5}$. A three-layer SiLU MLP projects each intent state to the teacher feature space. The projected rows are aligned one-to-one with the eight teacher intent targets I_t^* using cosine distance, with stop-gradient applied to the targets. The decoder-space states $H_{I,t}^{L_{\text{tap}}}$, rather than their projected teacher-space representations, are used as the recovered intent by the remaining decoder layers.

Intent-mismatch training. For updates with $\alpha = 0$, we construct mismatched intent states by cyclically shifting the recovered intent representations within the batch. Starting from the intent-alignment layer, the remaining decoder blocks are evaluated once with the original intent and once with the mismatched intent. We optimize

$$\mathcal{L}_{\text{mis}} = \max(0, m + \mathcal{D}_{\text{right}} - \mathcal{D}_{\text{swap}}),$$

where $\mathcal{D}_{\text{right}}$ and $\mathcal{D}_{\text{swap}}$ denote the combined downstream action and grounding losses, and $m = 0.05$. The mismatched donor representations are detached so that gradients do not propagate through the donor examples.

Grounding self-conditioning and deployment. Clean teacher targets are used only as supervision and are never supplied as decoder inputs. A no-gradient forward pass first estimates clean decoder-space visual and textual grounding states. These estimates are detached, re-noised at the sampled flow time, and used as grounding inputs to the gradient pass. During inference, the visual and textual states are carried across denoising steps using the same self-conditioning convention. The teacher VLM, target representations, alignment projections, and intent-mismatch branch are used only during training. The deployed policy receives the original VLA inputs and internally forms intent and grounding states together with the action stream.

A.6 TRAINING DETAILS

Optimization and learning-rate schedules. We follow the official fine-tuning implementations of GR00T-N1.7 (NVIDIA, 2026) and $\pi_{0.5}$ (Black et al., 2025a; Physical Intelligence, 2025). For each backbone, the baseline and INDI variant use the same benchmark-specific training budget. For GR00T-N1.7, we use AdamW with a learning rate of 10^{-4} . We train for 60,000 steps with a global batch size of 480 on RoboCasa Kitchen and for 20,000 steps with a global batch size of 1,536 on Bridge. For $\pi_{0.5}$, we use AdamW with a maximum gradient norm of 1.0 and linearly warm up the learning rate for 1,000 steps to 5×10^{-5} , which is held constant for the remainder of training. We use the same benchmark-specific batch sizes and training steps as in the corresponding GR00T-N1.7 experiments. All models are trained with bfloat16 computation, while loss values are accumulated in float32. The global batch sizes are distributed across three GPUs without gradient accumulation.

Loss configuration. We set $\lambda_I = \lambda_V = \lambda_R = 0.5$, $\lambda_{\text{mis}} = 0.1$, and the intent-mismatch margin to $m = 0.05$. The weights of the visual and textual grounding losses are linearly increased from zero to their final values during the first 1,000 training steps. The action and intent-alignment losses are active from the beginning of training. We use a context-dropout probability of 0.5.

Backbone-specific training configurations. For both backbones, we freeze the pretrained vision-language module and optimize the action decoder together with the newly introduced INDI parameters. The teacher targets are defined over an approximately 1.6-second behavior window, corresponding to $H = 8$ steps on Bridge and $H = 32$ steps on RoboCasa Kitchen. For GR00T-N1.7, the decoder retains its native 40-step output sequence, while the action and auxiliary losses are applied to the benchmark-valid segment of 8 steps on Bridge and 32 steps on RoboCasa Kitchen. For $\pi_{0.5}$, we retain its native 10-step action horizon and mask the action loss to the valid action dimensions, excluding dimensions introduced only by zero padding. The teacher behavior window is defined independently of the backbone-native policy horizon. We cache the keys and values of the frozen $\pi_{0.5}$ prefix during training.

Checkpoint selection. On RoboCasa Kitchen, we evaluate the final checkpoint obtained after 60,000 training steps. On Bridge, we evaluate checkpoints at 5,000, 10,000, 15,000, and 20,000 steps using two validation rollout seeds and apply the same selection procedure to both backbones.

Controlled ablation protocol. The mechanism, policy-level teacher, and intent-target-source ablations use GR00T-N1.7 on SimplerEnv-Bridge. Unless stated otherwise, all variants use the

same demonstrations, student architecture, optimization budget, checkpoint-selection procedure, and fixed evaluation run as the controlled analyses in [Section 4.4](#). Only the component or target source identified by each ablation is changed.

A.7 COMPUTATIONAL RESOURCES

All experiments are conducted on a single node with three NVIDIA B200 GPUs, each with 183 GB of memory. Teacher targets are generated once offline and reused across all subsequent policy-training runs. We summarize parameter counts, inference time per policy query, and benchmark-specific training time in [Table 7](#). All inference-time measurements are obtained on a single NVIDIA B200 GPU.

Table 7: Modification cost across VLA backbones. We report checkpoint-tensor parameter counts, inference time per policy query, and end-to-end training time. Inference is measured on a single NVIDIA B200 GPU, while training uses three NVIDIA B200 GPUs.

Method	Parameters	Inference time (ms)	Bridge training (h)	RoboCasa training (h)
GR00T-N1.7	3.455B	56.1 (1.00 $\times$)	~ 17	~ 17
+INDI	3.502B (+46.4M, +1.3%)	61.5 (1.10 $\times$)	~ 19	~ 19
$\pi_{0.5}$	3.617B	24.4 (1.00 $\times$)	~ 23	~ 23
+INDI	3.640B (+23.1M, +0.64%)	29.7 (1.21 $\times$)	~ 29	~ 26

B ADDITIONAL ABLATIONS AND DIAGNOSTICS

Unless stated otherwise, the following controlled analyses use GR00T-N1.7 on a fixed SimplerEnv-Bridge evaluation run shared across all variants. The main benchmark tables separately aggregate three evaluation runs. This section provides the exact control configurations, diagnostic protocols, full quantitative analyses, and additional readouts underlying [Section 4.4](#). Results already reported in [Tables 6a](#) and [6b](#) and [Fig. 3](#) are referenced directly rather than duplicated.

B.1 SUPERVISION AND CAPACITY CONTROLS

We isolate whether the improvement comes from teacher-derived intent, visual and textual grounding supervision, an endpoint-informed latent, or additional latent capacity. All variants use the same GR00T-N1.7 backbone, Bridge demonstrations, optimization budget, and evaluation protocol. The groundings-only control retains the visual and textual grounding streams, $\mathcal{L}_V$, $\mathcal{L}_R$, and grounding self-conditioning while removing the intent queries and intent supervision. The future-supervision control retains the latent and grounding streams but replaces I_t^* with an endpoint visual representation. The free-latent control retains the latent queries, bottleneck context routing, context dropout, visual and textual grounding streams, alignment objectives, and self-conditioning while removing both $\mathcal{L}_I$ and $\mathcal{L}_{\text{mis}}$. The intent-only control retains teacher-intent alignment, bottleneck context routing, context dropout, and action-only intent-mismatch training while removing the visual and textual grounding streams.

Table 8: Configuration of supervision and capacity controls. Task-level success rates are reported in [Table 6a](#).

Variant	Latent target	V/R groundings	Latent queries
Groundings only	–	Yes	No
Future supervision	Endpoint visual	Yes	Yes
Free latent	–	Yes	Yes
Intent only	Teacher intent	No	Yes
INDI	Teacher intent	Yes	Yes

As shown in [Table 6a](#), the non-intent controls reach at most 68.0%, whereas intent only reaches 76.0%. Teacher-derived intent therefore provides a 14.5 percentage-point gain over the action-

Table 9: **Supervision and capacity controls on SimplerEnv-Bridge.** V and R denote the visual and textual grounding streams. All variants share the backbone, budget, and a fixed evaluation run. Success rates (%).

Variant	V	R	Spoon	Carrot	Stack	EP-Basket	Avg.
GR00T-N1.7	–	–	82.0	68.0	66.0	30.0	61.5
+ Groundings only	✓	✓	88.0	82.0	38.0	32.0	60.0
+ Future supervision	✓	✓	84.0	70.0	56.0	62.0	68.0
+ Free latent	✓	✓	78.0	76.0	48.0	26.0	57.0
+ Intent only	–	–	86.0	84.0	70.0	64.0	76.0
+ Intent, V	✓	–	90.0	88.0	72.0	74.0	81.0
+ Intent, R	–	✓	90.0	86.0	70.0	72.0	79.5
INDI (Ours)	✓	✓	90.0	78.0	74.0	100.0	85.5

only baseline and an 8.0 percentage-point gain over future supervision. Adding visual and textual groundings further increases average success to 85.5%, showing that they complement rather than replace intent supervision.

Each grounding stream also contributes on its own, raising average success from 76.0% to 81.0% with the visual stream and 79.5% with the textual stream. Their joint effect (+9.5) exceeds the sum of the individual gains (+8.5), and is concentrated on EP-Basket, where either stream alone reaches roughly 73% but both together reach 100.0%. The textual stream helps despite aligning only weakly with its paired target (Table 14), consistent with it acting as a downstream realization rather than a standalone readout.

B.2 INTENT-DEPENDENCE MECHANISM ABLATIONS

We ablate the two training mechanisms introduced to make downstream decoding depend on recovered intent: bottleneck context routing and intent-mismatch training. The full model samples $\alpha \sim \text{Bernoulli}(0.5)$, requiring action and grounding rows to obtain VLA context through the intent rows when $\alpha = 0$, and applies $\mathcal{L}_{\text{mis}}$ to encourage lower downstream loss under matched than mismatched intent. We compare the full model with variants that remove the context bottleneck, the mismatch objective, or both.

Table 10: **Ablation of intent-dependence mechanisms on SimplerEnv-Bridge.** All variants use GR00T-N1.7, the same training budget, and the same fixed evaluation run. The context-bottleneck ablation allows action and grounding rows to access the VLA context throughout training, while the mismatch ablation sets $\lambda_{\text{mis}} = 0$.

Variant	Spoon	Carrot	Stack	EP-Basket	Avg.
INDI	90.0	78.0	74.0	100.0	85.5
w/o context bottleneck	88.0	76.0	60.0	56.0	70.0
w/o $\mathcal{L}_{\text{mis}}$	92.0	80.0	30.0	94.0	74.0
w/o both	86.0	72.0	58.0	46.0	65.5

Removing the context bottleneck reduces average success from 85.5% to 70.0%, while removing intent-mismatch training reduces it to 74.0%. Removing both mechanisms further lowers success to 65.5%. Thus, both mechanisms contribute substantially and complement one another: bottleneck routing encourages downstream decoding to obtain contextual information through the recovered intent, while $\mathcal{L}_{\text{mis}}$ encourages that dependence to be sensitive to the intent’s specific content rather than merely to the presence of the latent pathway.

B.3 FUNCTIONAL ROLE AND STRUCTURE OF RECOVERED INTENT

We examine whether the recovered representation is used by the action decoder, whether it retains behavior objective and execution progress across different action-sequence realizations, how this

information propagates through decoder depth, and whether objective-specific edits affect closed-loop behavior.

The action decoder relies on recovered intent. At the intent-alignment layer, we replace all eight recovered intent states with zero vectors while preserving the observation, instruction, proprioceptive state, action noise, grounding states, and remaining decoder computation.

Table 11: **Dependence on recovered intent.** The zero-intent intervention replaces all intent states at the alignment layer while preserving all other policy inputs. Both conditions use the same evaluation run on SimplerEnv-Bridge.

Intent state	Spoon	Carrot	Stack	EP-Basket	Avg.
Recovered intent	90.0	78.0	74.0	100.0	85.5
Zero intent	58.0	44.0	40.0	40.0	45.5

As shown in Table 11, zeroing recovered intent reduces average success from 85.5% to 45.5%, below the 61.5% action-only baseline. The recovered state is therefore not merely an auxiliary alignment target but a representation used by downstream action decoding.

Intent represents behavior objectives across domains. We collect the eight intent-row hidden states immediately before the alignment layer from 580 real demonstration segments and 202 simulation rollout segments. Each segment is represented by the flattened $K_I d_D$ -dimensional recovered intent state. We reduce the representations to 50 dimensions using PCA and fit three shrinkage-LDA axes from the four Bridge task labels. The two-dimensional task-discriminative projection is shown in Figure 3a. Raw-space one-nearest-neighbor classification achieves 92.0% task purity, and episode-disjoint shrinkage LDA achieves 86.2% accuracy. Randomly permuting the task labels gives 31.9% mean accuracy across 20 permutations, with a maximum of 41.4%. A classifier trained only on real-demonstration intents classifies simulation-rollout intents with 63.9% to 80.7% accuracy across tasks. These results show that objective information persists across episodes and transfers from real demonstrations to simulation rollouts.

Intent captures progress shared across related manipulation tasks. The four Bridge tasks manipulate different objects and target relations but share an approach–grasp–transport–place structure. We divide each episode into early, middle, and late intervals and fit a phase-discriminative projection using labels pooled across tasks. As shown in Figure 3b, segments from different tasks organize by execution stage. Three-way phase prediction achieves 77.8% five-fold cross-validation accuracy, compared with a 33.3% chance level. Task purity within the phase plane is 36.6%, while task identity remains readable within the early, middle, and late intervals at 94.0%, 90.0%, and 90.0%, respectively. We also fit a continuous progress direction with ridge regression and evaluate it in a leave-one-task-out setting. The held-out correlations are 0.62 for Spoon, 0.70 for Carrot, 0.72 for Stack, and 0.69 for EP-Basket, compared with an in-sample correlation of 0.88.

Table 12: **Objective and task-family-shared progress structure in recovered intent.** The supervision control applies the same probe to the free-latent variant, which receives identical context access but no teacher-intent supervision.

Property	Evaluation	Result
<i>Behavior objective</i>		
Objective identity	Raw-space task 1-NN purity	92.0%
Objective generalization	Episode-disjoint task LDA	86.2%
Supervision control	Same probe, free latent (no $\mathcal{L}_I$)	55.7%
Cross-domain transfer	Train on real, test on simulation	63.9–80.7%
Objective retention	Task purity in early/middle/late phases	94.0/90.0/90.0%
<i>Shared execution progress</i>		
Phase readability	Three-way phase prediction	77.8%
Task leakage	Task purity within phase plane	36.6%
Cross-task transfer	Leave-one-task-out progress correlation	0.62–0.72

Together, Figure 3 and Table 12 show complementary objective and progress structure in the recovered representation. The task-discriminative structure captures which object-level objective is being pursued, while the phase-discriminative structure captures progress shared by this family of manipulation tasks. We interpret the latter as a task-family-shared progress coordinate rather than a universal progress axis for arbitrary robot skills.

Intent captures objective beyond action-sequence realization. We distinguish intent from skill by operationalizing skill as the executed action sequence. The action sequence describes how a behavior is realized, whereas intent denotes the local objective that the behavior serves. To test whether intent merely summarizes the executed sequence, we compare progress-matched episode pairs under two opposing conditions: the same objective realized by dissimilar sequences, and different objectives realized by similar sequences. We use episode-disjoint discovery and test splits, with all centering statistics and pair-selection thresholds estimated from discovery episodes.

As shown in Figure 6, teacher intent targets separate by objective on held-out episodes, while a matched supervised projection of local action snippets yields weak objective separation. More importantly, the projection-free comparison shows that recovered-intent similarity remains higher for the same objective with dissimilar sequences than for different objectives with similar sequences (0.150 vs. -0.017), with a gap of 0.167 and a 95% episode-bootstrap confidence interval of $[0.089, 0.245]$. Teacher targets show the same ordering (0.470 vs. 0.158; gap 0.312, 95% CI $[0.253, 0.375]$). Thus, the representation preserves what the behavior is meant to accomplish across changes in how it is executed, distinguishing intent from a latent code that merely summarizes the action sequence.

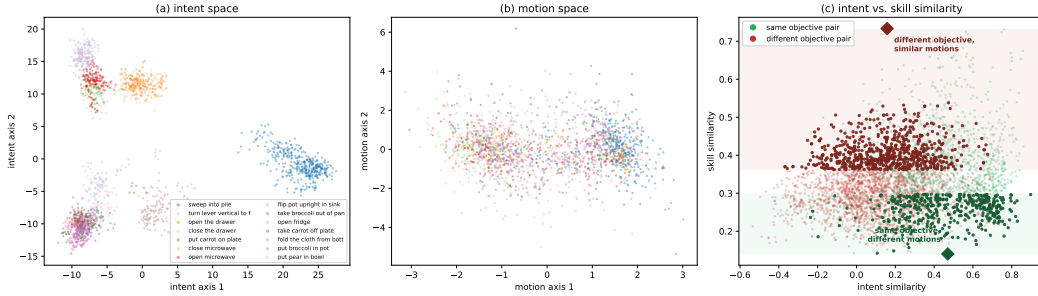


Figure 6: Intent captures objective beyond action-sequence realization. We operationalize skill as the executed action sequence. (a) Teacher intent targets separate 14 behavior objectives on held-out episodes. (b) A matched supervised projection of local action snippets yields weak objective separation. (c) In a projection-free comparison of progress-matched episode pairs, the same objective realized by dissimilar action sequences remains more similar in intent space than different objectives realized by similar sequences. Light points show all eligible pairs, dark points show the selected contrast sets, and diamonds mark their means. Recovered policy intent exhibits the same ordering.

Task information propagates from intent into the grounding streams. A one-time edit at the intent-alignment layer may be overwritten if later blocks reconstruct the original objective from other token streams. We therefore measure one-nearest-neighbor task purity for the state, action, visual-grounding, textual-grounding, and intent rows throughout the 32 decoder blocks.

Table 13: Task information across decoder rows and depth. The table summarizes task information before and after the intent-alignment layer.

Decoder rows	Blocks 0–15	Blocks 19–31
Intent	0.63 → 0.93	0.77 ± 0.05
Visual grounding	Near chance	Up to 0.95
Textual grounding	Near chance	Up to 0.97
Action and state	Near chance	Near chance

Intent-row task purity peaks at 0.93 immediately before the alignment layer. After alignment, task information appears in the visual and textual grounding rows, reaching 0.95 and 0.97, while the action and state rows remain near chance. This pattern motivates editing both intent and grounding rows throughout the decoder tail.

Editing objective-specific coordinates changes closed-loop behavior. Zeroing intent establishes that the pathway is necessary but does not isolate the effect of its objective content. We therefore edit only the task-discriminative coordinates while preserving the remaining representation. Let $g \in \{I, G\}$ index the edited token group, where I contains the intent rows and $G = V \oplus R$ contains the visual and textual grounding rows. Let $h_{g,k} \in \mathbb{R}^{p_g}$ denote the flattened representation after decoder block k . For task t , we compute the block-wise centroid

$$\mu_{g,k}^{(t)} = \mathbb{E}[h_{g,k} \mid \text{task} = t]$$

and form a centered matrix from the four task centroids. The columns of $U_{g,k}$ are the right singular vectors spanning the resulting between-task subspace. For an injected task t_d , we replace only the coordinates inside this subspace:

$$h'_{g,k} = h_{g,k} + U_{g,k} \left[U_{g,k}^\top \mu_{g,k}^{(t_d)} - U_{g,k}^\top h_{g,k} \right].$$

This sets the task-discriminative coordinates to those of the injected objective while preserving the orthogonal component. We apply the edit to the intent and grounding groups after every decoder block from 16 through 24, without modifying the action or state rows. Task centroids and bases are estimated from separate rollouts under environment seeds disjoint from intervention evaluation. Each same-objective, cross-objective, and Gaussian-noise condition uses the same intervention schedule and is evaluated over 50 episodes. Donors are captured from separate rollouts, so the same-objective condition also receives an injection of equal strength and differs from the cross-objective condition only in donor content.

The resulting closed-loop success matrix is reported in Table 6b. Same-objective interventions retain 84.5% success on average, compared with 45.2% for cross-objective interventions, while Gaussian corruption reduces success to 1.0%. The dependence on which objective is injected, together with the contrast against unstructured noise, shows that the intent-mediated pathway carries content rather than acting as an undifferentiated gate.

Intent geometry reflects task relatedness. The uneven effects of cross-objective intervention raise the question of whether the recovered intent space reflects which tasks are related. We therefore compare centered cosine similarity between recovered-intent centroids for the four SimplerEnv evaluation tasks. As shown in Figure 7, Spoon and Carrot form the closest pair, Stack is more distinct, and EP-Basket is the most separated from the other tasks. This organization is consistent with Spoon and Carrot sharing a closely related tabletop manipulation structure, whereas EP-Basket differs in both its objective and scene. The result provides a descriptive representation-level notion of task relatedness that may help explain why some cross-objective transfers are more compatible than others. Because scene and objective differences are not independently controlled, we do not interpret the similarity as a causal measure of transfer.

Editing stage-specific coordinates changes which stage is executed. We repeat the procedure with $U_{g,k}$ spanning the between-phase subspace, estimated from early, middle, and late centroids pooled across tasks using the projection of Figure 3b. All other aspects of the protocol are unchanged, so the two interventions differ only in which subspace is replaced. Forcing an early-stage donor reduces average success to 7.3%, and forcing a late-stage donor reduces it to 5.5%, both approaching the 1.0% obtained under Gaussian corruption. The phase-matched control, which injects a donor from the same task at the receiver’s own stage, retains 50%, 70%, and 32% on Spoon, Carrot, and Stack. The gap between this control and the uninjected policy reflects the cost of clamping the intent stream over the window, which varies by task.

As shown in Figures 8 to 10, rollouts under stage-forced injection do not fail incoherently but execute behavior consistent with the injected stage. Early forcing leaves the policy executing approach behavior without transitioning to grasp: it hovers over the target on Carrot, displaces the spoon without closing the gripper on Spoon, and fumbles at the green block without initiating the stack on Stack. Late forcing produces the opposite pattern, skipping the grasp and executing placement. This is clearest on Spoon, where the policy moves to the towel and presses down with an empty

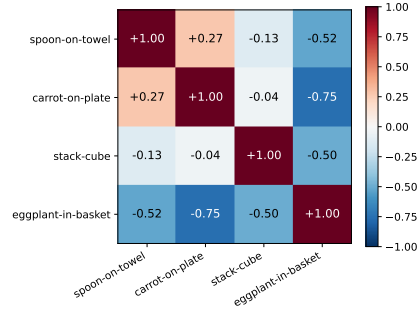


Figure 7: **Task relatedness in recovered-intent space.** Each entry reports centered cosine similarity between the recovered-intent centroids of two SimplerEnv tasks. Spoon and Carrot form the closest pair, while EP-Basket is the most separated from the other tasks.

gripper. Since both conditions apply injections of equal strength and differ only in donor content, the contrast is attributable to the stage information carried by the intent pathway rather than to perturbation magnitude.

Phase-matched control grasps the spoon, carries it, places it on the towel



Early-forced pokes and displaces the spoon, never forms a grasp



Late-forced presses the towel with an empty gripper (place without object)

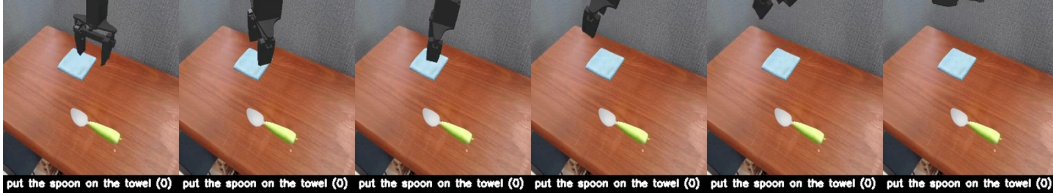
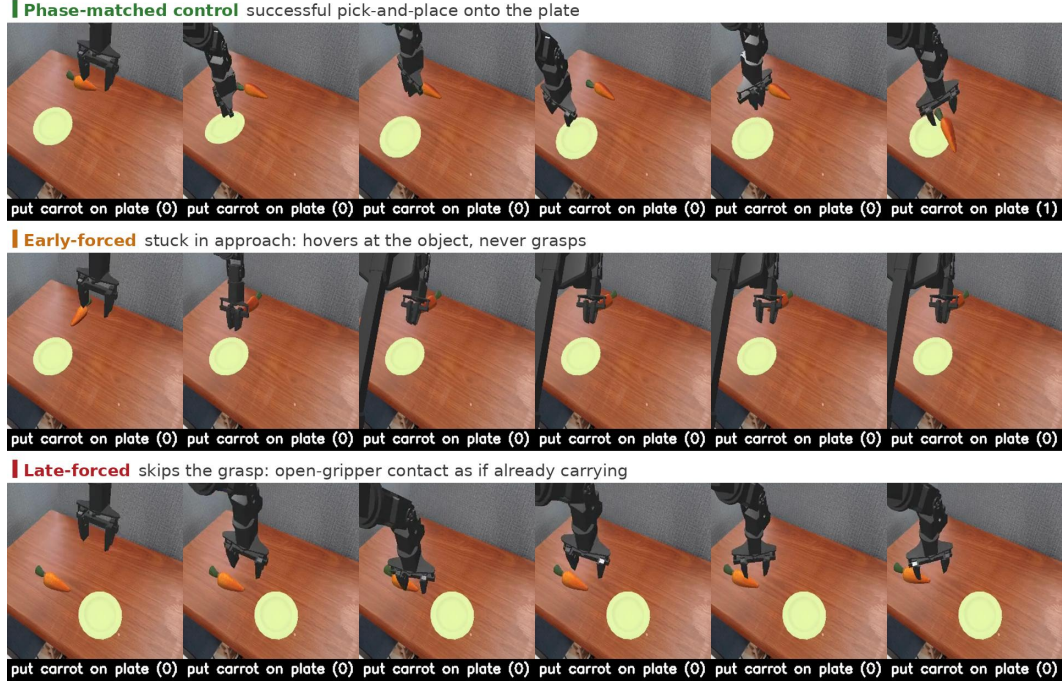


Figure 8: **Stage-forced intervention on Spoon.** Rows show phase-matched, early-forced, and late-forced injection. Early forcing leaves the policy displacing the spoon without ever closing the gripper. Late forcing skips the grasp entirely: the policy moves to the towel and presses down with an empty gripper, executing placement as if it were already carrying the object.

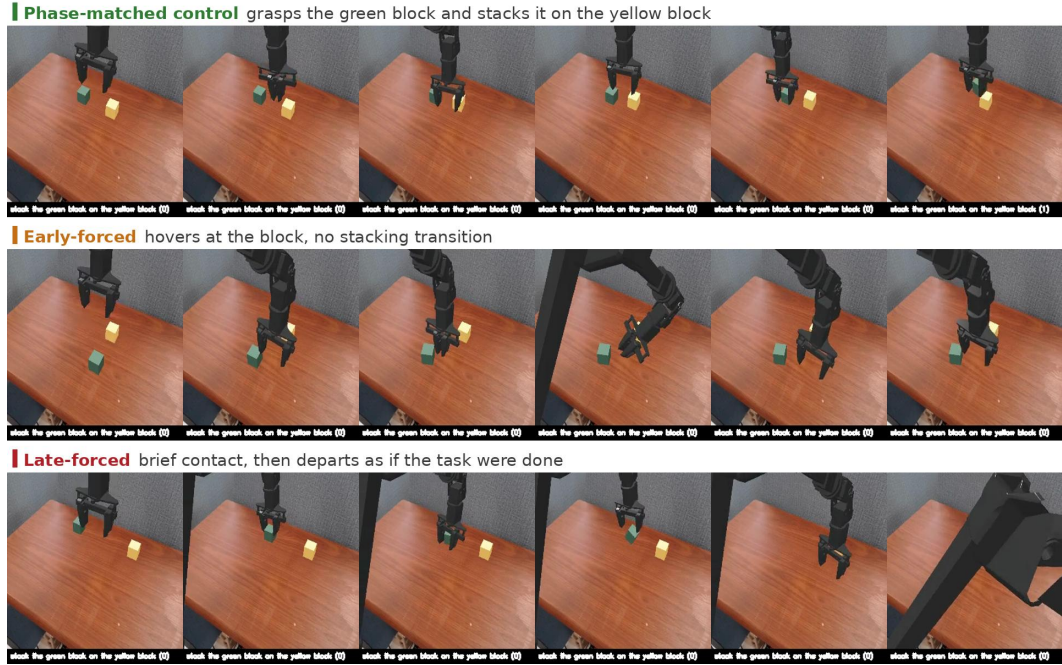
B.4 VISUAL AND TEXTUAL READOUT OF INTENT

Section B.1 identifies which supervision improves policy performance, whereas this subsection localizes the resulting behavior information within the decoder. We freeze the trained policy for all readout experiments. Let

$$\hat{I}_t = P_I(H_{I,t}^{L_{\text{tap}}}), \quad \hat{V}_t = P_V(H_{V,t}^{L_{\text{out}}}), \quad \hat{R}_t = P_R(H_{R,t}^{L_{\text{out}}})$$



1481 Figure 9: **Stage-forced intervention on Carrot**. Early forcing leaves the policy hovering over the
 1482 carrot without transitioning to grasp. Late forcing produces open-gripper contact, as if the object
 1483 were already being transported.
 1484



1508 Figure 10: **Stage-forced intervention on Stack**. Early forcing leaves the policy fumbling at the
 1509 green block without initiating the stack. Late forcing produces brief contact followed by departure,
 1510 as if the task were already complete.
 1511

denote the projected intent and the final visual and textual grounding predictions. We evaluate them using nearest-neighbor retrieval and separately trained readout decoders while keeping the policy frozen.

Embedding-space readout. We extract $\hat{V}_t$, $\hat{R}_t$, V_t^* , and R_t^* for approximately 10,000 held-out segments. We subtract the dataset mean before computing cosine similarity and exclude retrieval candidates from the same episode.

Table 14: **Embedding-space readout of the visual and textual groundings.** Correct cosine measures centered similarity to the paired target. Shuffled cosine uses randomly permuted targets. Median rank is zero-indexed.

Readout	Correct cosine	Shuffled cosine	Margin	Top-1	Top-10	Median rank
$\hat{V}_t \rightarrow V_t^*$	0.650	0.003	0.647	72.6%	95.3%	0
$\hat{R}_t \rightarrow R_t^*$	0.158	-0.004	0.161	1.5%	5.0%	1211

The visual grounding provides a highly readable endpoint representation, retrieving the paired target at rank one in 72.6% of held-out segments and within the top ten in 95.3%. The textual grounding is more similar to its paired target than to shuffled targets, although its standalone nearest-neighbor geometry is less concentrated. Purpose semantics are most compactly represented in $\hat{I}_t$, while the textual grounding rows provide a downstream semantic realization used during joint decoding.

Visual grounding depends on intent. We repeat the visual readout after zeroing recovered intent or replacing it with recovered intent from another task while preserving the remaining policy inputs.

Table 15: **Effect of intent intervention on visual grounding.**

Intent condition	Centered margin	Top-1 retrieval
Recovered intent	0.647	72.6%
Zero intent	0.513	59.1%
Cross-task intent	0.393	41.8%

Visual readout quality decreases from recovered to zero to cross-task intent. The predicted visual outcome therefore depends on the content supplied through the intent pathway.

Visual-outcome retrieval. We retrieve actual endpoint frames using $\hat{V}_t$. For the intervention montage, the retrieval bank includes the designated current-task and injected-task endpoints.

The examples in Figure 11 show that recovered intent preserves the current object, scene, and result-relation, while cross-task intent redirects the predicted outcome toward the injected behavior.

Learned visual and textual decoding. We additionally train lightweight decoders to test whether the representations can be converted directly into an endpoint image and a functional-purpose statement. The visual decoder maps the 16 ordered visual slots to the endpoint RGB image using a transformer bottleneck and convolutional upsampling. The policy is frozen, and the decoder is trained on visual targets V_t^* and endpoint observations. On held-out data, reconstruction from V_t^* obtains a changed-region L_1 error of 0.0819, compared with 0.3042 for copying the current observation. For textual readout, we compare the final textual grounding $\hat{R}_t$ with the projected intent $\hat{I}_t$. The intent decoder is trained on approximately 9,900 unmodified intent-statement pairs using an episode-disjoint split. We report deterministic object F1, state-change verb F1, and ROUGE-L.

The recovered intent is substantially more readable than the final textual grounding, improving object F1 from 0.314 to 0.693 and verb F1 from 0.195 to 0.432. This localizes the semantic bottleneck primarily in recovered intent while retaining the textual grounding as a downstream representation used during joint decoding.

The visual and textual readouts change consistently under intent intervention. These results support the intended hierarchy of INDI, in which intent stores a compact behavior-level representation and visual outcome and textual purpose provide complementary observable realizations.

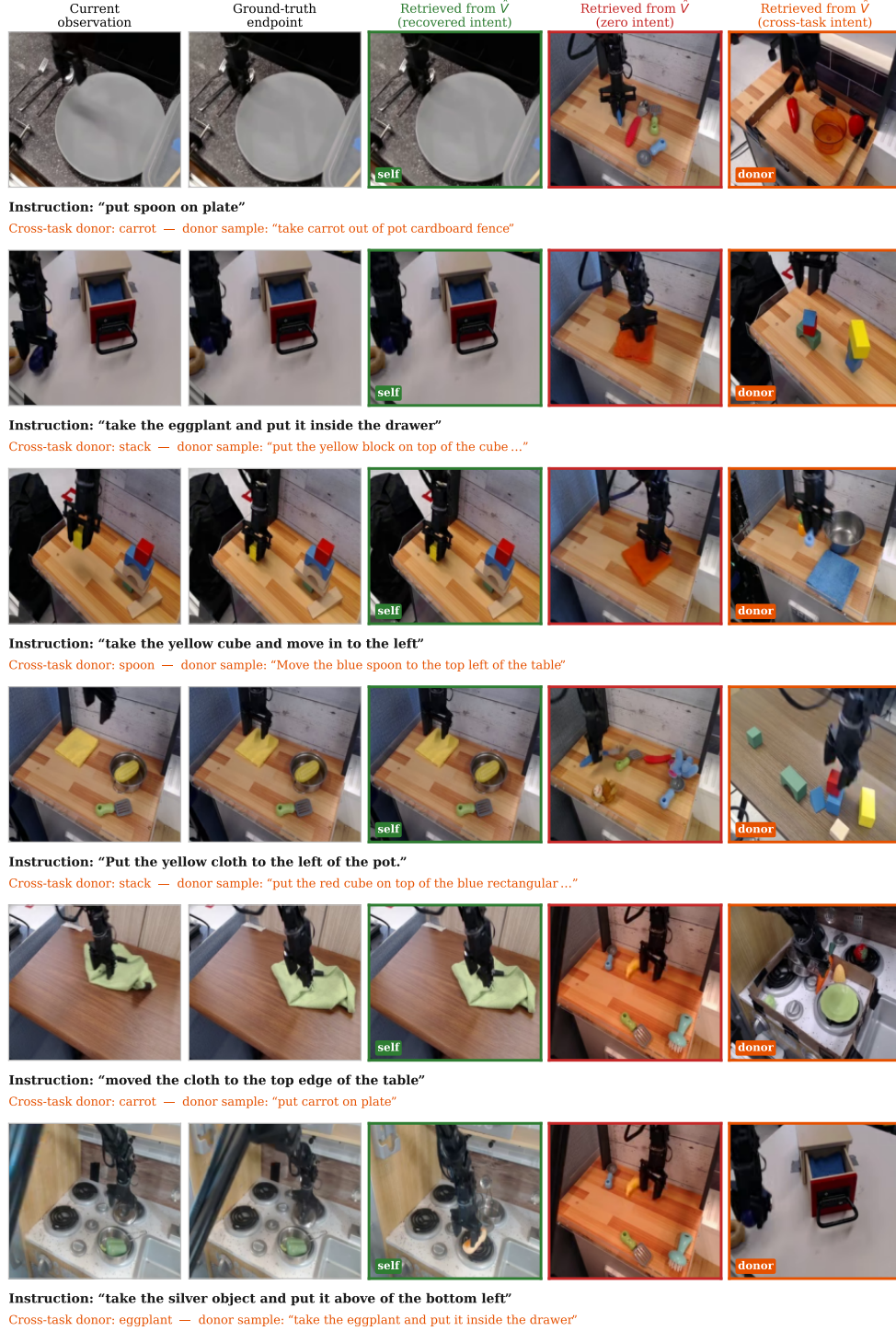
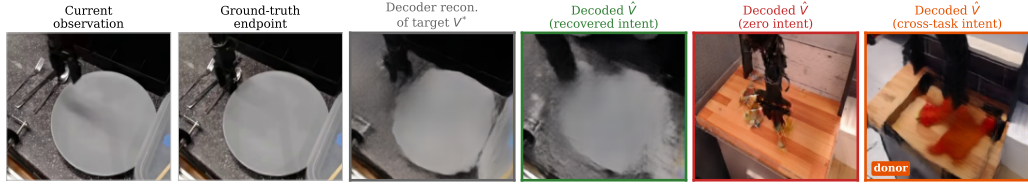


Figure 11: **Non-parametric visual-outcome readout.** Each row shows the current observation, ground-truth endpoint, and endpoint retrieved from $\hat{V}_t$ under recovered, zero, and cross-task intent. Recovered intent retrieves the current behavior outcome, while cross-task intent redirects retrieval toward the injected behavior. Across the evaluated pairs, recovered intent retrieves the current endpoint in 70.5% of cases. After cross-task replacement, the injected endpoint is retrieved in 78.6% of cases, while the original endpoint remains in only 0.1%.



Instruction: "put spoon on plate"

Cross-task donor: carrot

Ground-truth purpose: Position the spoon over the plate, aligning it for placement.

Decoded from recovered intent: The gripper is moving left and forward to locate and approach the spoon for grasping.

Decoded from zero intent: The gripper repositions itself to the left and upward, likely preparing to interact with the orange...

Decoded from donor intent: Lift the carrot out of the pot and move it away from the workspace.

Donor ground-truth purpose: Relocate the carrot from the pot to a position outside the cardboard fence, moving it leftward...



Instruction: "take the eggplant and put it inside the drawer"

Cross-task donor: stack

Ground-truth purpose: The gripper has securely grasped the blue object and is lifting it upward to prepare for placement...

Decoded from recovered intent: The gripper has securely grasped the green object and is lifting it upward, preparing to transfer it...

Decoded from zero intent: The gripper repositions itself to the left and upward, likely preparing to interact with the orange...

Decoded from donor intent: The gripper positions itself to grasp the yellow block by moving downward and rotating to align...

Donor ground-truth purpose: The gripper attempts to grasp the yellow block by moving downward and leftward while remaining open.



Instruction: "take the yellow cube and move it to the left"

Cross-task donor: spoon

Ground-truth purpose: The gripper moves the yellow cube leftward and downward, rotating slightly while maintaining a firm...

Decoded from recovered intent: The gripper is moving the yellow block leftward and upward to reposition it for placement.

Decoded from zero intent: The gripper repositions itself to the left and upward, likely preparing to interact with the orange...

Decoded from donor intent: The gripper is moving the blue spoon to the left side of the table while rotating to align it...

Donor ground-truth purpose: The gripper is moving the blue spoon leftward and upward while rotating slightly to reposition it...



Instruction: "Put the yellow cloth to the left of the pot."

Cross-task donor: stack

Ground-truth purpose: The gripper positions itself over the yellow cloth to prepare for grasping it.

Decoded from recovered intent: The gripper positions itself to grasp the yellow towel in preparation for moving it to the left of...

Decoded from zero intent: The gripper repositions itself to the left and upward, likely preparing to interact with the orange...

Decoded from donor intent: Place the blue block on top of the yellow cube to complete the stacking task.

Donor ground-truth purpose: The gripper places the red cube on top of the blue rectangular block.



Instruction: "moved the cloth to the top edge of the table"

Cross-task donor: carrot

Ground-truth purpose: Lift and reposition the cloth toward the top edge of the table using upward and backward motion.

Decoded from recovered intent: The gripper repositions itself to prepare for moving the cloth to the left side of the table.

Decoded from zero intent: The gripper repositions itself to the left and upward, likely preparing to interact with the orange...

Decoded from donor intent: The gripper is moving the carrot to the right and upward, rotating it while rotating counterclockwise...

Donor ground-truth purpose: The gripper is positioning itself over the carrot inside the box to prepare for grasping.

Figure 12: **Learned visual and textual readout of intent.** Each row shows the current observation, ground-truth endpoint, visual reconstruction from V_t^* , and visual outcomes decoded from $\hat{V}_t$ under recovered, zero, and cross-task intent. The text below each row shows the ground-truth purpose and statements decoded from recovered, zero, and injected intent. Recovered intent produces an outcome and purpose consistent with the current behavior, while cross-task intent redirects both modalities toward the injected behavior.

Table 16: **Textual-purpose readout.** The oracle row decodes the teacher textual target. The remaining rows decode the predicted textual grounding and recovered intent.

Readout representation	Object F1	Verb F1	ROUGE-L
R_t^* (oracle target)	0.998	1.000	0.933
$\hat{R}_t$	0.314	0.195	0.255
$\hat{I}_t$	0.693	0.432	0.380

B.5 TEACHER QUALITY AND INTENT-TARGET GEOMETRY

We examine whether intent distillation benefits from an arbitrary teacher, or whether the teacher target itself must have particular structure. We repeat the controlled `SimplerEnv-Bridge` training while changing only the teacher cache, keeping the demonstrations, student architecture, optimization budget, checkpoint selection, and evaluation protocol fixed.

Teacher choice matters. Table 17 shows that teacher choice substantially changes downstream performance. Qwen3.5-9B (Qwen Team, 2026) reduces average success to 54.5%, below the 61.5% action-only baseline, whereas Qwen3-VL-8B-Instruct (Bai et al., 2025) reaches 73.5% and Cosmos-Reason2-8B reaches 85.5%. Since Cosmos-Reason2-8B (NVIDIA, 2025a) is obtained by physical-AI post-training of Qwen3-VL-8B-Instruct, this pair provides a controlled comparison of the effect of teacher-side post-training.

Table 17: **Controlled teacher swap on `SimplerEnv-Bridge`.** All variants use the same GR00T-N1.7 student, demonstrations, optimization budget, and fixed evaluation run; only the teacher cache is changed.

Teacher	Spoon	Carrot	Stack	EP-Basket	Avg.
No intent distillation	82.0	68.0	66.0	30.0	61.5
Qwen3.5-9B	84.0	78.0	40.0	16.0	54.5
Qwen3-VL-8B-Instruct	90.0	98.0	34.0	72.0	73.5
Cosmos-Reason2-8B	90.0	78.0	74.0	100.0	85.5

The difference is visible in the intent targets. We characterize each teacher’s I^* targets on the same 17,288 valid Bridge samples. The *angular budget* measures the mean cosine distance of L2-normalized targets from their mean direction, and therefore how much sample-dependent directional variation is available to the cosine-alignment objective. The *effective rank* summarizes the spectrum of the centered target distribution, while the *within-episode share* measures the fraction of centered variance that occurs within episodes rather than between them.

Table 18: **Intent-target geometry across teachers.** All statistics use the same valid Bridge samples ($n = 17,288$). Bridge Avg. reports the corresponding controlled policy performance.

Teacher	Angular budget	Eff. rank	Within-ep. share	Bridge Avg.
Qwen3.5-9B	0.0027	139.4	0.308	54.5
Qwen3-VL-8B-Instruct	0.0098	7.2	0.693	73.5
Cosmos-Reason2-8B	0.0327	3.4	0.723	85.5

Qwen3.5 produces an almost degenerate alignment signal: its angular budget is only 0.0027, so a nearly constant prediction can satisfy much of the cosine-alignment objective, and only 30.8% of its residual variation occurs within episodes. Qwen3-VL instead exhibits substantially more sample-dependent and state-resolved variation and recovers a 12.0-percentage-point gain over the action-only baseline. Cosmos preserves the high within-episode share while increasing the angular budget by $3.3\times$ over Qwen3-VL, accompanied by a further 12.0-point downstream gain. The high effective rank of Qwen3.5 therefore does not indicate a useful teacher by itself; it describes the structure of a very small residual signal.

Across the three teachers, angular budget and within-episode share increase monotonically with downstream success. Given the small number of teachers, we treat this as a diagnostic condition rather than a predictive law: useful intent supervision requires a non-degenerate target whose variation resolves moment-to-moment behavior state rather than primarily static episode identity. The causal interpretation is limited to the Qwen3-VL–Cosmos pair, where physical-AI post-training is the controlled teacher-side change.

B.6 INTENT-TARGET SOURCE ABLATION

The default intent target is extracted from the intermediate hidden states over the multimodal teacher-input span $\mathcal{E}_t$. We compare this evidence-side target with the generated-response target $I_t^{\star, \text{resp}}$, which pools the intermediate hidden states over the complete teacher response $\mathcal{Y}_t$, including its reasoning and final functional-purpose statement. Both variants use the same teacher, $K_I = 8$ target slots, student architecture, grounding targets, optimization budget, and evaluation protocol. The comparison therefore isolates whether policy supervision is more effective when intent is represented during multimodal evidence processing or after the teacher has synthesized that evidence into a generated response.

Table 19: **Intent-target source ablation on SimplerEnv-Bridge.** Evidence-side and generated-response targets use the same teacher layer and $K_I = 8$ contiguous-region pooling.

Intent target	Spoon	Carrot	Stack	EP-Basket	Avg.
Evidence-side input span $\mathcal{E}_t$	90.0	78.0	74.0	100.0	85.5
Generated-response span $\mathcal{Y}_t$	76.0	86.0	76.0	80.0	79.5

The generated-response target obtains 79.5% average success, compared with 85.5% for the evidence-side multimodal target. Although the generated-response target substantially outperforms the 61.5% action-only baseline, the evidence-side target provides a further 6.0 percentage-point gain. This result indicates that both target sources provide useful behavior-level supervision, while directly distilling the representation formed during multimodal evidence processing is more effective than using the representation formed after reasoning and response generation.

C ADDITIONAL RESULTS

C.1 FULL QUANTITATIVE RESULTS

We provide complete RoboCasa Kitchen results to further examine the generality of INDI across VLA backbones and manipulation tasks. Table 20 reports the per-task success rates summarized by task category in the main text. The table compares INDI with controlled GR00T-N1.7 and $\pi_{0.5}$ baselines, the GR00T-N1.7 future-supervision variant, and off-the-shelf GR00T checkpoints trained with larger demonstration sets.

The future-supervision variant exhibits substantially larger per-task variation than INDI, improving markedly on some tasks (e.g., Open Double Door, Turn Off Stove) while degrading on others where the remaining variants are near ceiling (e.g., Turn On Sink Faucet, Coffee Press Button). Its category and overall averages therefore reflect a redistribution of per-task performance rather than a uniform improvement.

C.2 QUALITATIVE RESULTS

We provide representative real-world rollouts for all four tasks in Figures 13 and 14. Each filmstrip shows the third-person view together with the head and two wrist-camera views. The complete rollout videos are provided on the project page.

Table 20: **Full RoboCasa Kitchen per-task results.** Success rates are reported across all 24 tasks. Controlled GR00T-N1.7 variants and $\pi_{0.5}$ results report mean $\pm$ sample standard deviation over three evaluation runs. The GR00T G_{3000} columns provide single-value data-scale references. Bold marks the best controlled result within each backbone in the category-summary rows.

Task	GR00T					$\pi_{0.5}$	
	N1.6 G_{3000}	N1.7 G_{3000}	N1.7 G_{100}	N1.7 + future supervision G_{100}	N1.7 + INDI G_{100}	Baseline	+ INDI
<i>Pick-and-Place</i>							
PnP from Cab to Counter	41.0	65.0	26.0 $\pm$ 16.0	44.7 $\pm$ 1.2	50.0 $\pm$ 4.0	18.7 $\pm$ 2.3	10.7 $\pm$ 10.1
PnP from Counter to Cab	47.5	60.0	37.3 $\pm$ 3.1	35.3 $\pm$ 5.0	58.7 $\pm$ 10.3	18.7 $\pm$ 2.3	23.3 $\pm$ 7.6
PnP from Counter to Microwave	19.0	30.0	22.0 $\pm$ 5.3	57.3 $\pm$ 5.8	28.0 $\pm$ 0.0	6.7 $\pm$ 2.3	6.7 $\pm$ 2.3
PnP from Counter to Sink	46.0	60.0	54.7 $\pm$ 11.7	29.3 $\pm$ 11.4	53.3 $\pm$ 10.3	10.0 $\pm$ 7.2	16.0 $\pm$ 4.0
PnP from Counter to Stove	63.2	60.0	46.7 $\pm$ 7.0	56.0 $\pm$ 6.0	50.7 $\pm$ 9.9	12.0 $\pm$ 6.9	8.0 $\pm$ 0.0
PnP from Microwave to Counter	24.5	19.0	23.3 $\pm$ 3.1	26.7 $\pm$ 2.3	32.0 $\pm$ 2.0	8.0 $\pm$ 0.0	8.0 $\pm$ 0.0
PnP from Sink to Counter	50.0	65.0	50.7 $\pm$ 8.3	64.7 $\pm$ 9.9	62.7 $\pm$ 3.1	24.7 $\pm$ 9.9	21.3 $\pm$ 2.3
PnP from Stove to Counter	54.5	65.0	54.7 $\pm$ 4.2	62.0 $\pm$ 3.5	62.7 $\pm$ 8.1	13.3 $\pm$ 2.3	28.7 $\pm$ 4.2
<i>Open-or-Close</i>							
Close Double Door	88.5	80.0	88.0 $\pm$ 7.2	60.0 $\pm$ 5.3	90.0 $\pm$ 2.0	57.3 $\pm$ 25.7	72.0 $\pm$ 6.9
Close Drawer	100.0	100.0	92.7 $\pm$ 8.1	68.0 $\pm$ 8.0	100.0 $\pm$ 0.0	77.3 $\pm$ 39.3	98.7 $\pm$ 2.3
Close Single Door	96.0	95.0	96.0 $\pm$ 4.0	89.3 $\pm$ 9.5	97.3 $\pm$ 3.1	73.3 $\pm$ 11.5	68.7 $\pm$ 1.2
Open Double Door	39.0	25.0	52.0 $\pm$ 5.3	94.0 $\pm$ 6.0	75.3 $\pm$ 3.1	32.0 $\pm$ 5.3	10.7 $\pm$ 2.3
Open Drawer	81.1	95.0	58.0 $\pm$ 8.7	88.0 $\pm$ 3.5	55.3 $\pm$ 5.8	42.7 $\pm$ 1.2	36.0 $\pm$ 14.4
Open Single Door	81.5	90.0	68.7 $\pm$ 3.1	82.7 $\pm$ 8.1	78.7 $\pm$ 7.0	48.0 $\pm$ 18.3	50.7 $\pm$ 6.1
<i>Others</i>							
Coffee Press Button	98.5	100.0	98.0 $\pm$ 2.0	32.0 $\pm$ 2.0	100.0 $\pm$ 0.0	20.0 $\pm$ 8.0	74.7 $\pm$ 6.1
Coffee Serve Mug	63.5	85.0	74.7 $\pm$ 9.0	73.3 $\pm$ 4.2	81.3 $\pm$ 7.0	25.3 $\pm$ 9.2	23.3 $\pm$ 9.9
Coffee Setup Mug	31.0	30.0	32.0 $\pm$ 10.6	79.3 $\pm$ 13.3	30.7 $\pm$ 1.2	14.7 $\pm$ 12.2	4.0 $\pm$ 0.0
Turn Off Microwave	96.0	95.0	99.3 $\pm$ 1.2	92.7 $\pm$ 4.2	99.3 $\pm$ 1.2	72.0 $\pm$ 16.0	90.7 $\pm$ 4.6
Turn Off Sink Faucet	93.5	100.0	93.3 $\pm$ 6.4	85.3 $\pm$ 7.6	98.0 $\pm$ 2.0	74.7 $\pm$ 19.7	89.3 $\pm$ 6.1
Turn Off Stove	31.0	25.0	27.3 $\pm$ 8.1	93.3 $\pm$ 5.8	28.7 $\pm$ 2.3	22.7 $\pm$ 22.0	16.7 $\pm$ 4.2
Turn On Microwave	91.5	95.0	89.3 $\pm$ 8.1	92.7 $\pm$ 4.6	93.3 $\pm$ 4.2	53.3 $\pm$ 2.3	72.7 $\pm$ 6.4
Turn On Sink Faucet	89.0	95.0	84.0 $\pm$ 8.0	18.7 $\pm$ 9.0	96.0 $\pm$ 4.0	33.3 $\pm$ 18.5	61.3 $\pm$ 12.9
Turn On Stove	76.5	85.0	72.0 $\pm$ 4.0	60.0 $\pm$ 3.5	70.7 $\pm$ 2.3	33.3 $\pm$ 11.5	41.3 $\pm$ 10.1
Turn Sink Spout	87.0	80.0	97.3 $\pm$ 3.1	93.3 $\pm$ 4.2	93.3 $\pm$ 5.8	45.3 $\pm$ 2.3	61.3 $\pm$ 6.1
Pick-and-Place	43.2	53.0	39.4 $\pm$ 1.0	47.0 $\pm$ 4.4	49.8 $\pm$ 2.3	14.0 $\pm$ 1.7	15.3 $\pm$ 1.4
Open-or-Close	81.0	80.8	75.9 $\pm$ 3.4	80.3 $\pm$ 2.3	82.8 $\pm$ 1.3	55.1 $\pm$ 7.4	56.1 $\pm$ 3.6
Others	75.8	79.0	76.7 $\pm$ 0.9	72.1 $\pm$ 2.0	79.1 $\pm$ 1.5	39.5 $\pm$ 0.9	53.5 $\pm$ 2.4
Average	66.2	70.8	64.1 $\pm$ 1.6	65.8 $\pm$ 0.8	70.3 $\pm$ 1.7	34.9 $\pm$ 2.0	41.4 $\pm$ 1.4

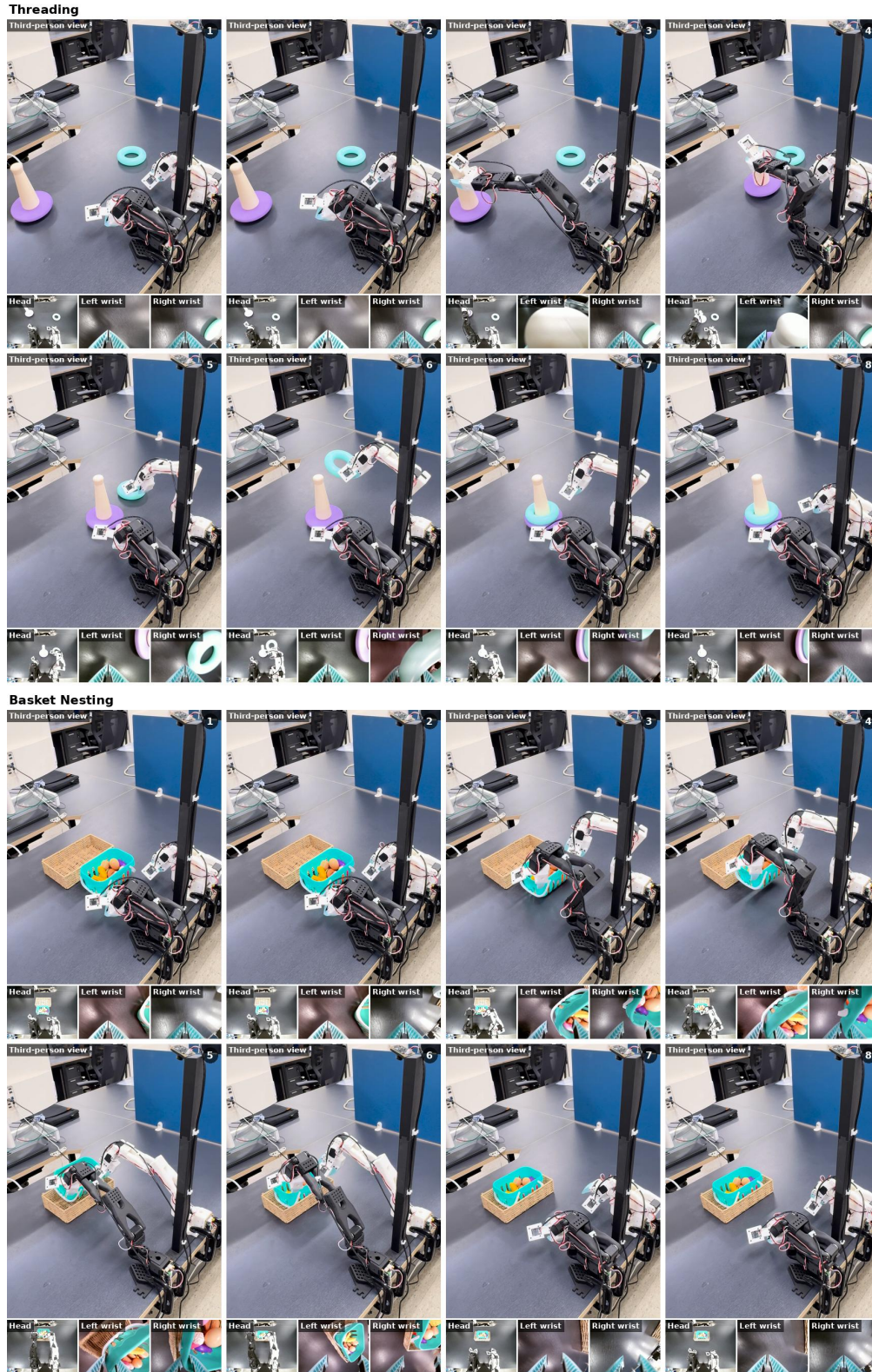


Figure 13: **Representative real-world rollouts on Threading and Basket Nesting.** The filmstrips show successful execution across the third-person, head, and wrist-camera views.

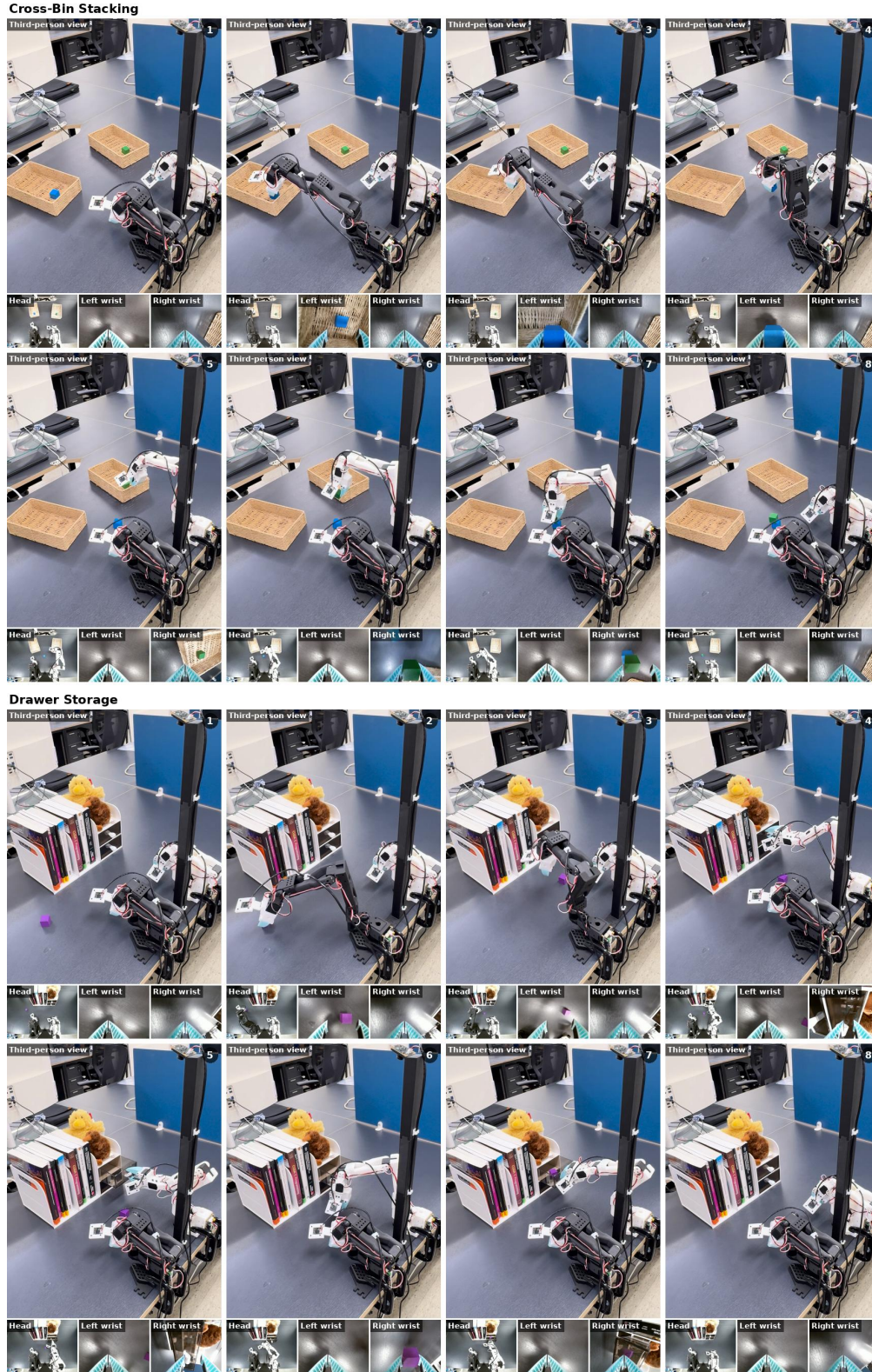


Figure 14: **Representative real-world rollouts on Cross-Bin Stacking and Drawer Storage.** The examples illustrate coordinated execution across the multiple stages required by the two longer-horizon tasks.

D LIMITATIONS AND FUTURE WORK

INDI inherits its notion of intent from a teacher VLM, so the semantics available to the policy are bounded by what that teacher understands about physical behavior. The bound is consequential rather than cosmetic. A teacher without embodied grounding yields targets with almost no sample-dependent direction, and the resulting policy falls below the action-only baseline (Tables 17 and 18). This property can be measured before training, but the framework still has no notion of an intent being wrong and no mechanism that revises a target when execution contradicts it. Removing this ceiling requires intent that is validated against execution outcomes or acquired through the policy’s own interaction rather than read from a fixed interpreter.

A second limit is that intent is recovered rather than chosen. The deployed policy reconstructs one deterministic intent at each query and immediately consumes it. Our interventions show that this state is addressable (Section B.3), yet the policy never uses that handle. It cannot hold a distribution over valid objectives when the stage is ambiguous, maintain an intent across a rollout, or accept a correction stated as what it should be trying to do. Treating intent as an interface for a planner, a human, or execution feedback is the extension we consider most consequential.

Finally, intent is read from single windows of demonstrations that succeeded. Manipulation objectives are nested, and failed or corrected episodes state the objective most explicitly while accounting for much of the experience a robot can collect. Hierarchical intent and intent distilled from unsuccessful behavior are natural next steps.

E USE OF LARGE LANGUAGE MODELS

We used large-language-model-based tools, including Claude Code and Codex CLI, to assist with code navigation, experiment orchestration, and language editing. All methodological decisions, implementation changes, experimental procedures, and reported results were reviewed and verified by the authors. The Cosmos-Reason2 teacher used to construct training targets is a component of the proposed method and is distinct from the tools used during manuscript and code preparation.